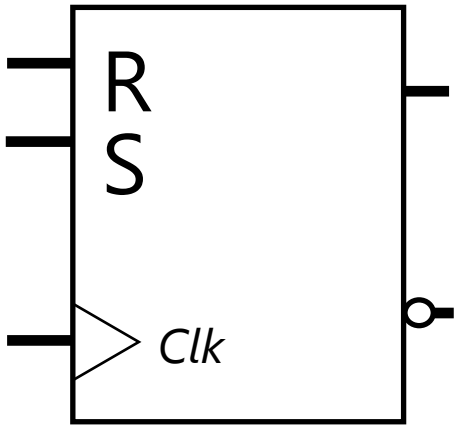
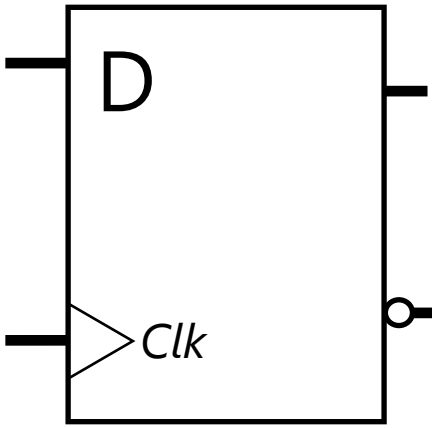
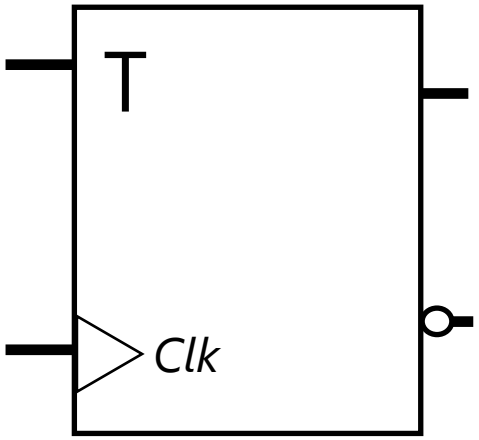
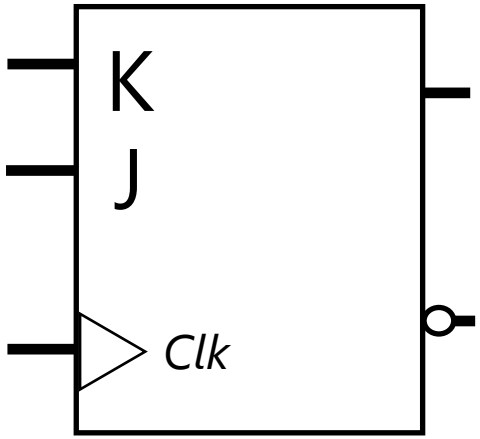
Flip-Flop

A single edge triggered latch









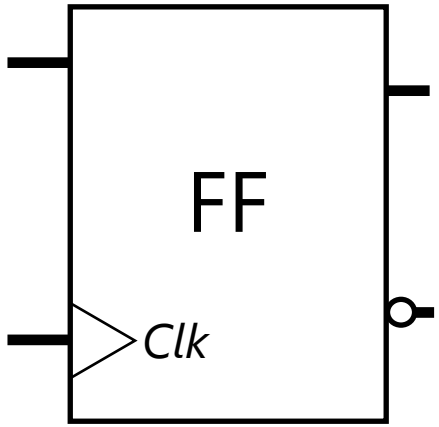
S	R	Q
0	0	Q_t
0	1	0
1	0	1
1	1	X

D	Q
0	0
1	1

T	Q
0	Q_t
1	Q'_t

J	K	Q
0	0	Q_t
0	1	0
1	0	1
1	1	Q'_t

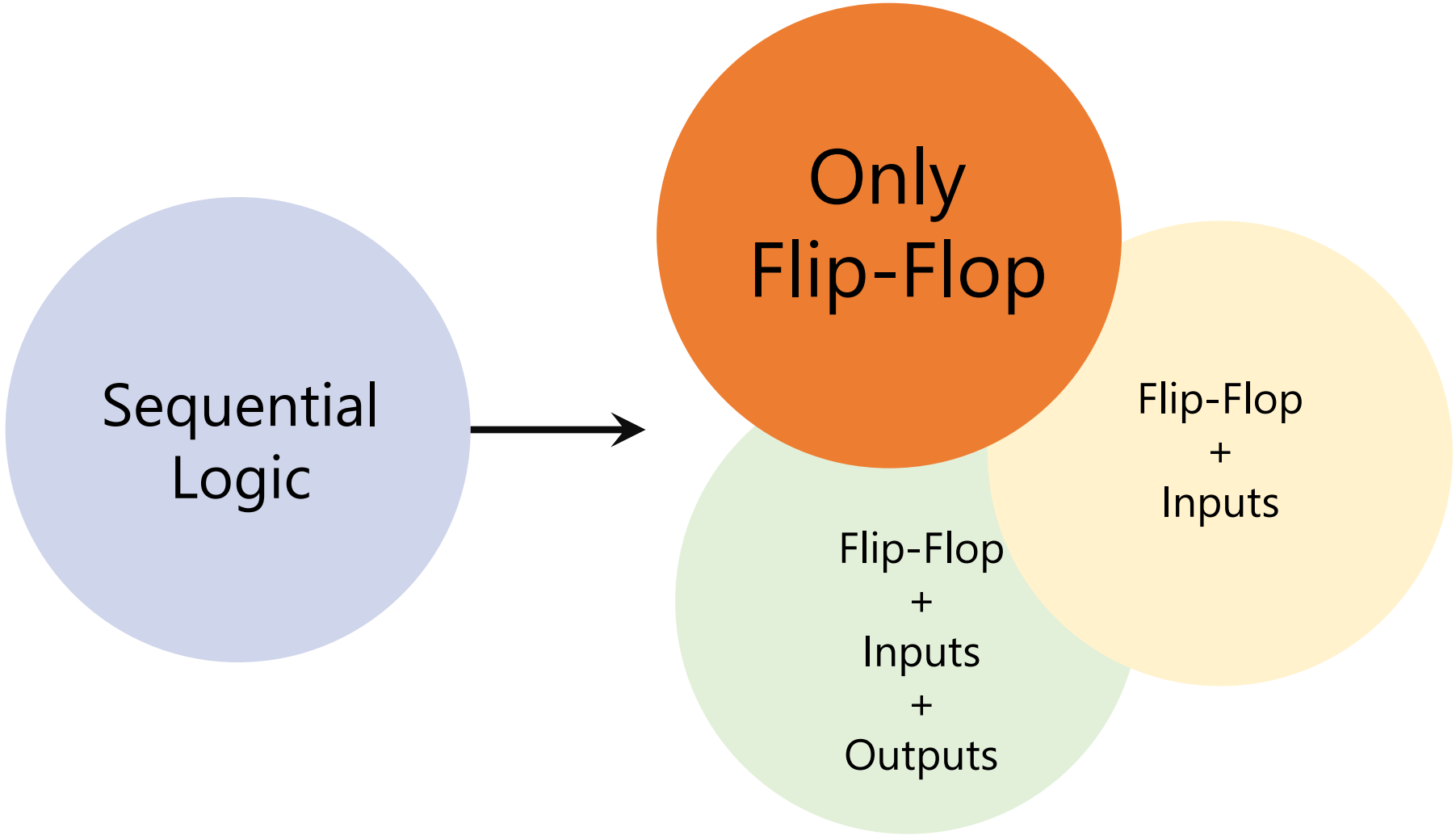
Single Edge
Positive



Analysis: Given a sequential circuit, show the behavior

vs.

Design: Given a behavior, build the sequential circuit


Sequential Logic
Only Flip-Flop
Flip-Flop + Inputs
Flip-Flop + Inputs + Outputs

Analysis (Recap)

0. Is the circuit sequential or combinational? Any FF or feedback → Sequential
1. What are the flip-flops? RS, D, T, JK, or mixed (e.g., 2 JK, 1 RS, …)
2. What are the state combinations? $2^{\#FF}$
3. Form "State" table:
   a) Columns: for each FF, two columns:
      - one for current state,
      - one for next state
   b) Rows: for each state combination
      - In total: $2^{\#FF}$
4. Fill the state table for next state columns based on:
   a) the current state
   b) the inputs to the FFs
5. Form State Transition Diagram
6. (Optional) Analyze paths and states in state transition diagram

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. Form the state (transition) diagram
3. Form the state table
4. Fill the state table
5. What type of storage (flip-flop)? RS, D, T, JK, or Mixed
6. Input (*excitation*) equations for each FF
7. Minimization of input (*excitation*) equations
8. Draw/Sketch Logic Circuit
9. (Optional) Test

Practice

Step Couter: Skip Every Other Number

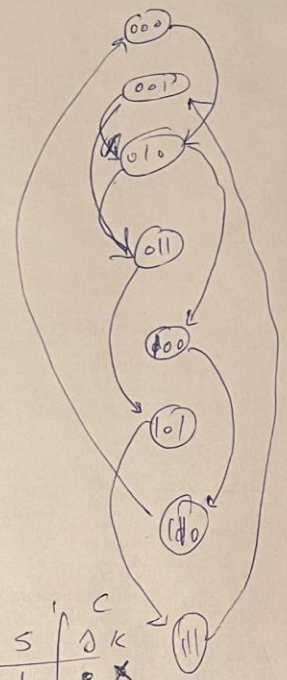
0 → 2 → 4 → 6

1 → 3 → 5 → 7

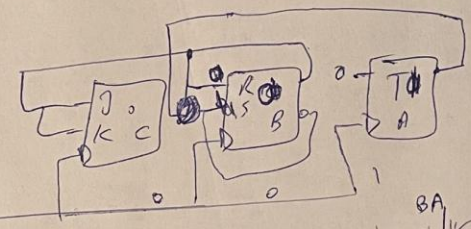
5

~~Loop Count Even~~ → 0 → 2 → 4 → 6

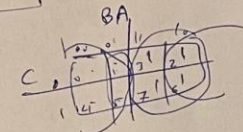
→ not → 1 → 3 → 5 → 7



Q(T)			Q(T+1)			T_A
C	B	A	C	B	A	T_A
0	0	0	0	0	0	hold 0
0	0	1	0	1	1	hold 0
0	1	0	0	0	0	hold 0
0	1	1	1	0	1	hold 0
1	0	0	1	1	0	hold 0
1	0	1	1	1	1	hold 0
1	1	0	0	0	0	hold 0
1	1	1	0	0	1	hold 0



Action	B R	B S	C J	C K
set	0	1	0	X
	0	1	0	X
	1	0	1	X
	1	0	1	X
	0	1	X	0
	0	1	X	0
	1	0	X	1
	1	0	X	1



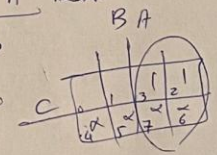
$T_A = 0$

$R_B = \Sigma(2,3,6,7) = B$

$S_B = \Sigma(0,1,4,5) = B'$

$J_C = \Sigma(2,3) + D(4-7) = B$

$K_C = \Sigma(6,7) + D(0-3) = B$



Design (Recap)

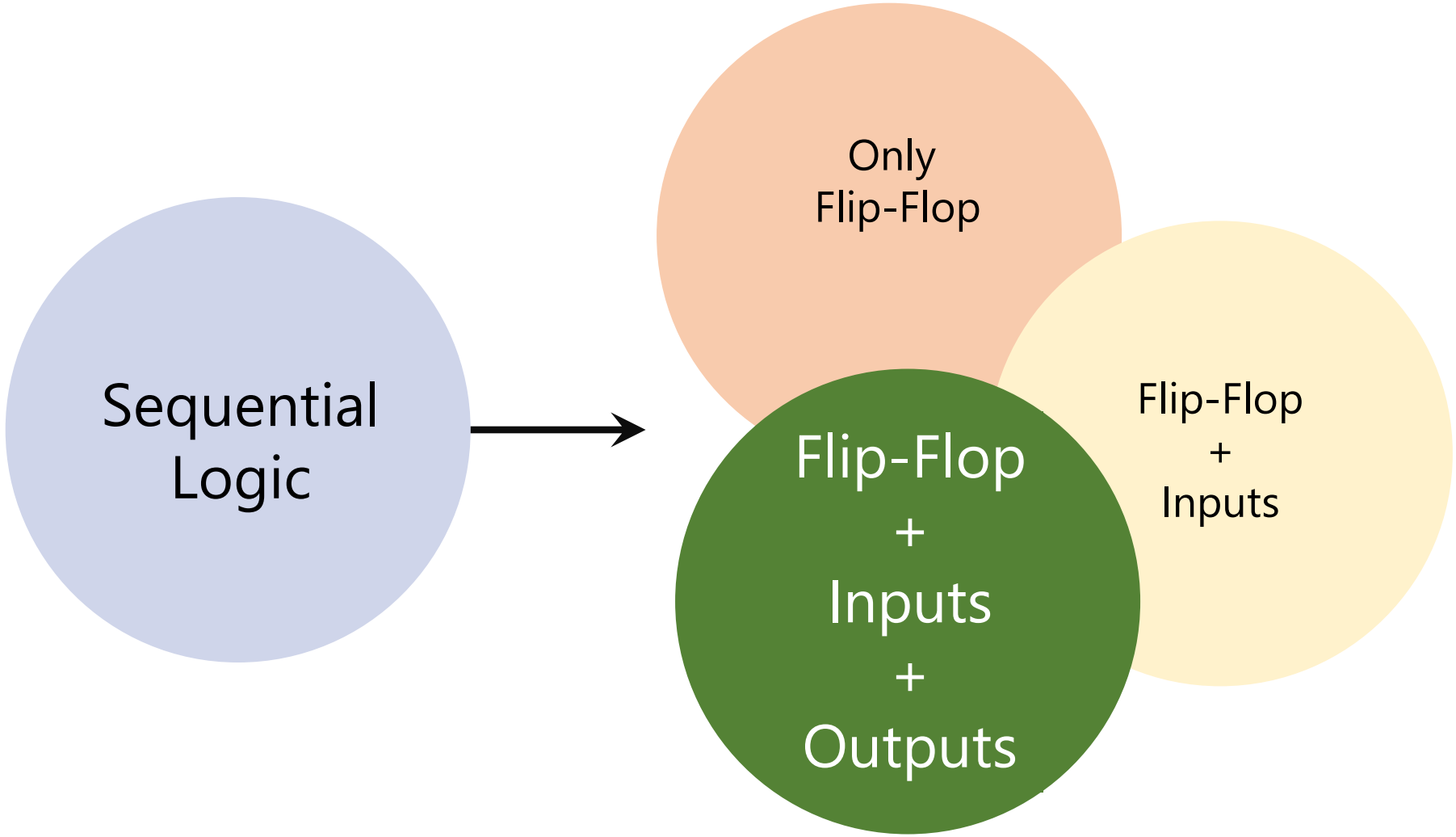
0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. Form the state (transition) diagram
3. Form the state table
4. Fill the state table
5. What type of storage (flip-flop)? RS, D, T, JK, or Mixed
6. Input (*excitation*) equations for each FF
7. Minimization of input (*excitation*) equations
8. Draw/Sketch Logic Circuit
9. (Optional) Test

Design (Advanced)

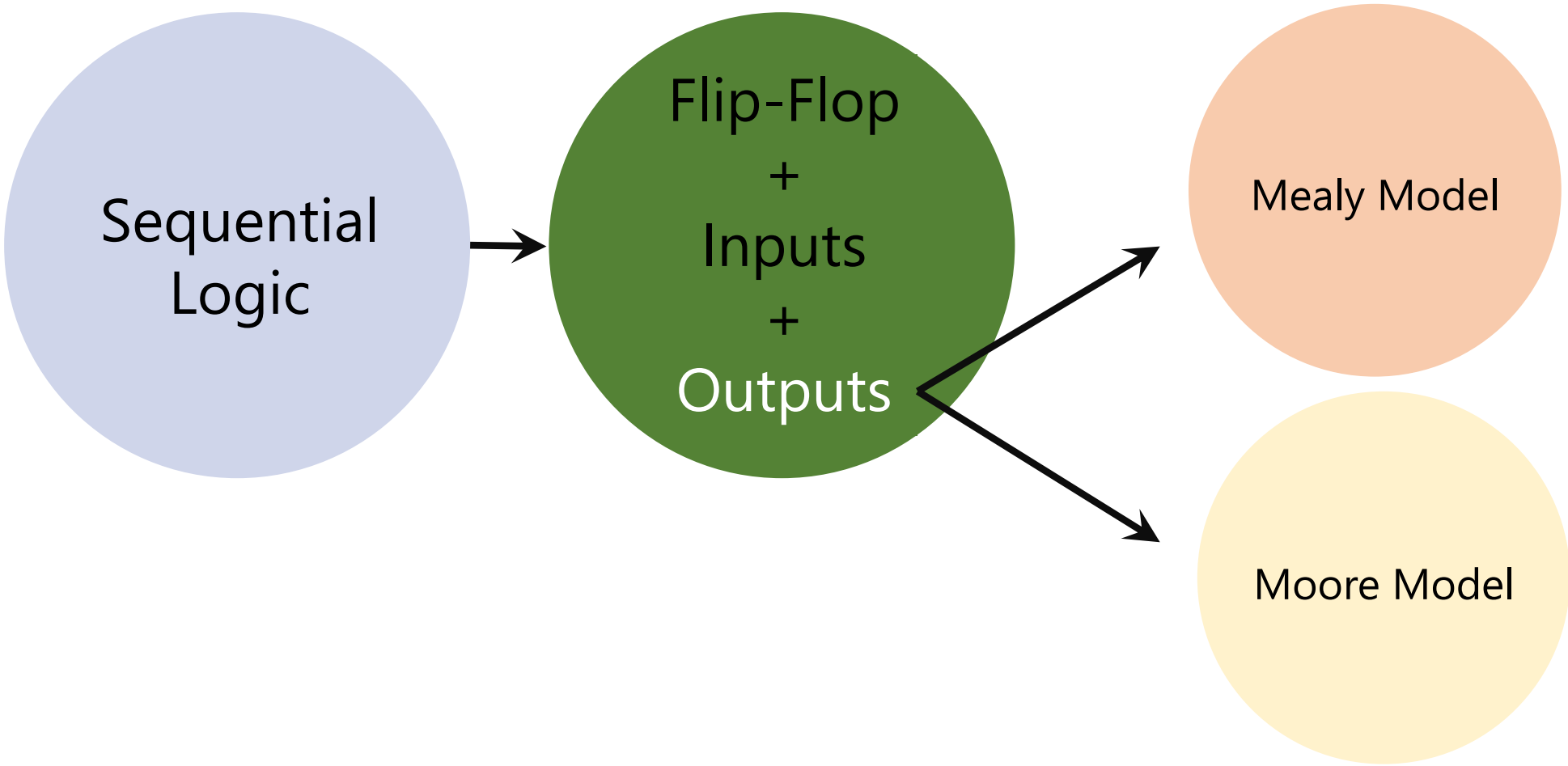
0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. Form the state (transition) diagram
2.1. State Reduction
3. Form the state table
4. Fill the state table
5. What type of storage (flip-flop)? RS, D, T, JK, or Mixed
6. Input (*excitation*) equations for each FF
7. Minimization of input (*excitation*) equations
8. Draw/Sketch Logic Circuit
9. (Optional) Test

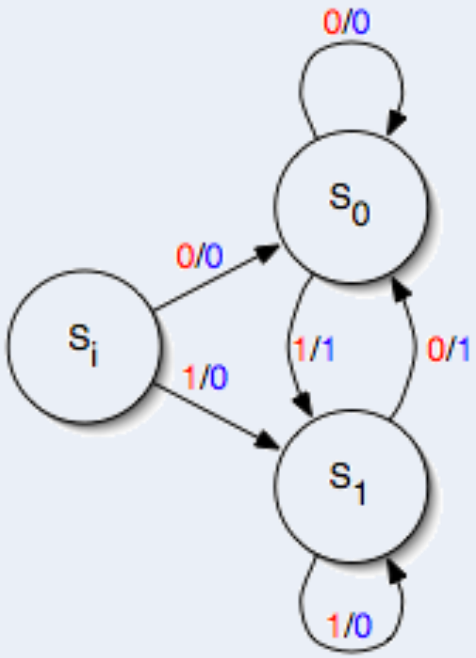
Theory of Automata

COMP-2140: Computer Languages, Grammars, and Translators


Sequential Logic
Only Flip-Flop
Flip-Flop + Inputs
Flip-Flop + Inputs + Outputs

Analysis
vs.
Design

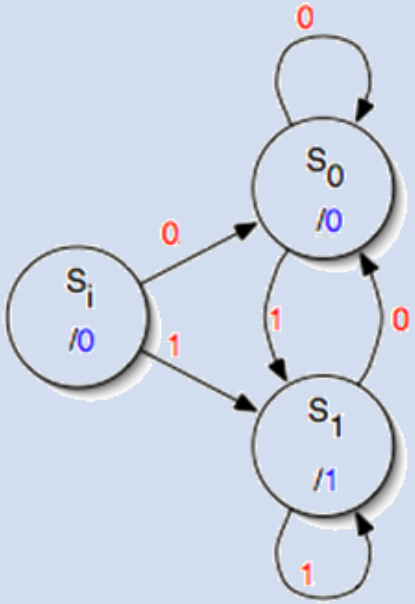

Sequential Logic
Flip-Flop
+
Inputs
+
Outputs
Mealy Model
Moore Model



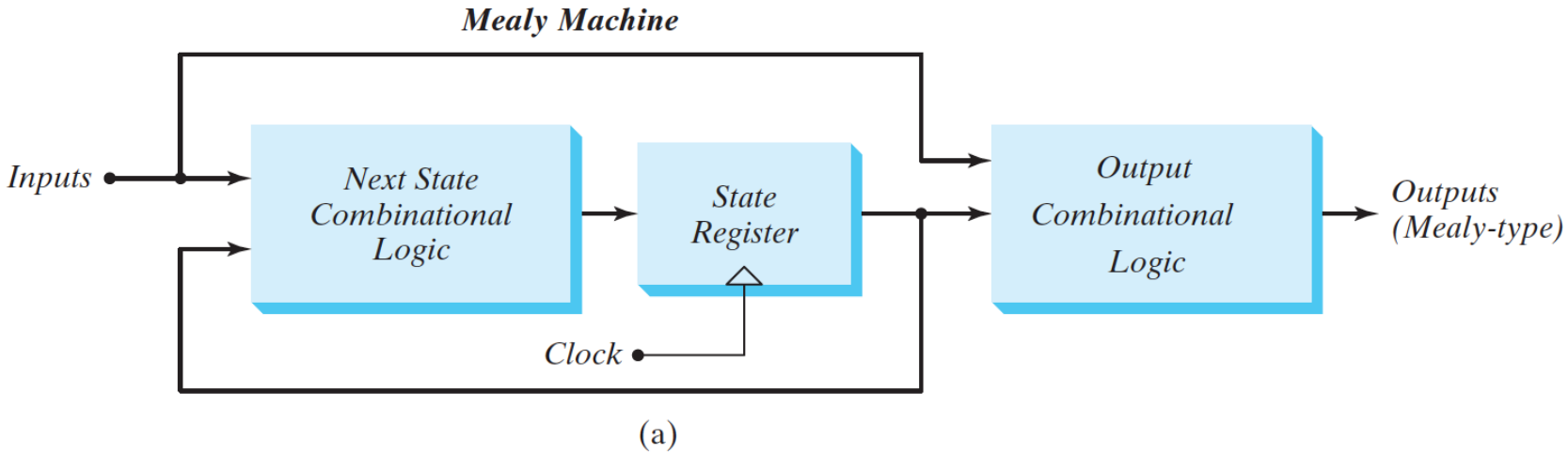
George H. Mealy
(1927 – 2010)
Mathematician and Computer Scientist
Invented Mealy Machine
Also a pioneer of modular programming

Outputs = Function(Current State, Inputs)

Edward Forrest Moore
(1925 – 2003)
Mathematician and Computer Scientist
Inventor of the Moore Machine
Also an early pioneer of artificial life



Outputs = Function(Current State~~, Inputs~~)



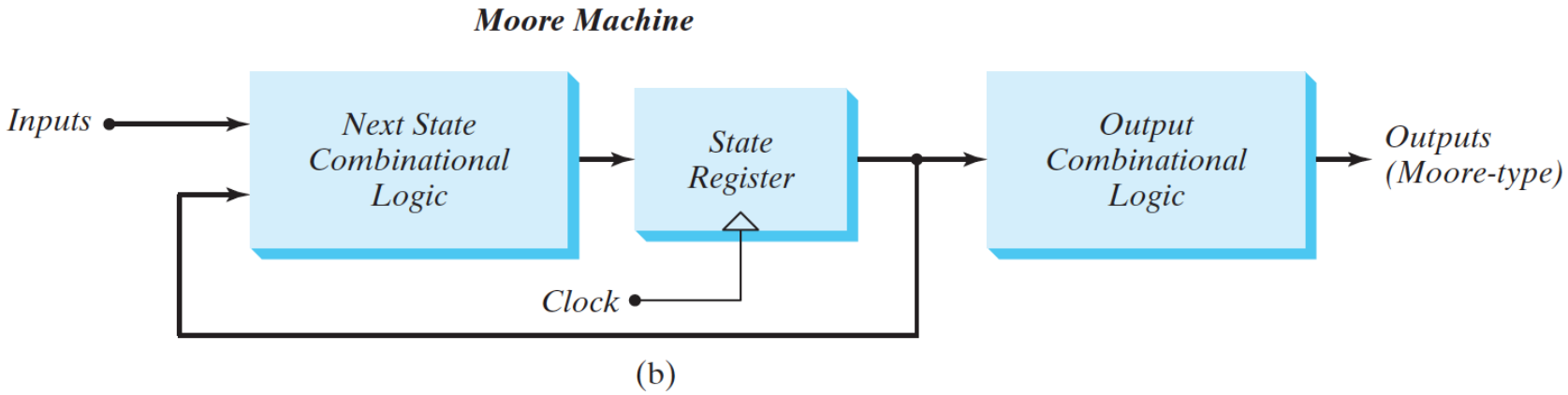
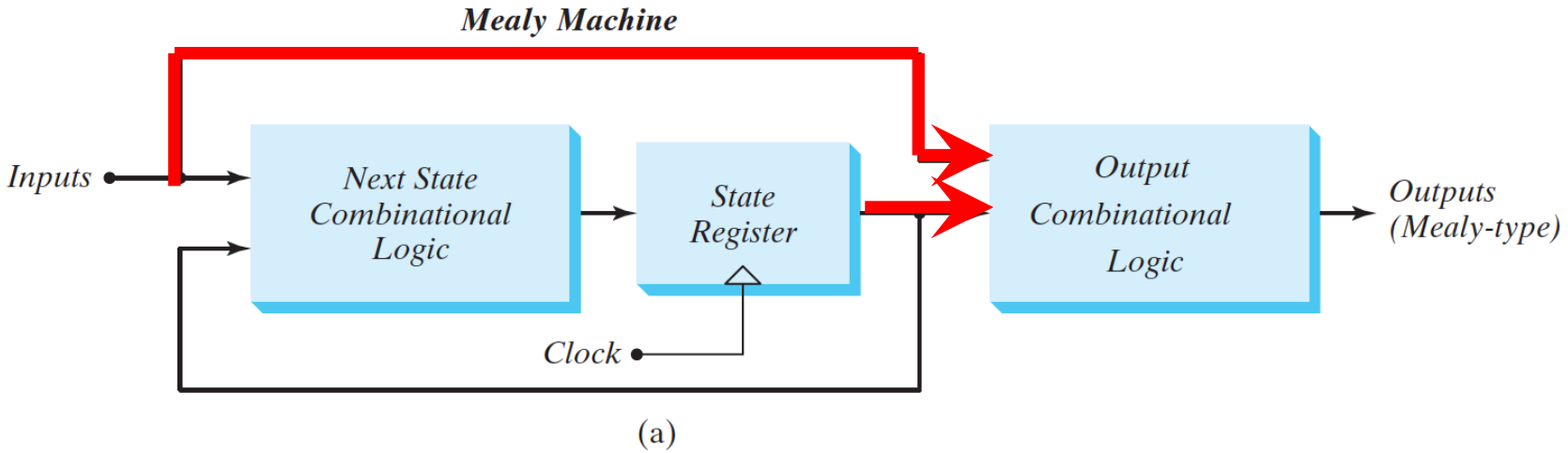


FIGURE 5.21
Block diagrams of Mealy and Moore state machines



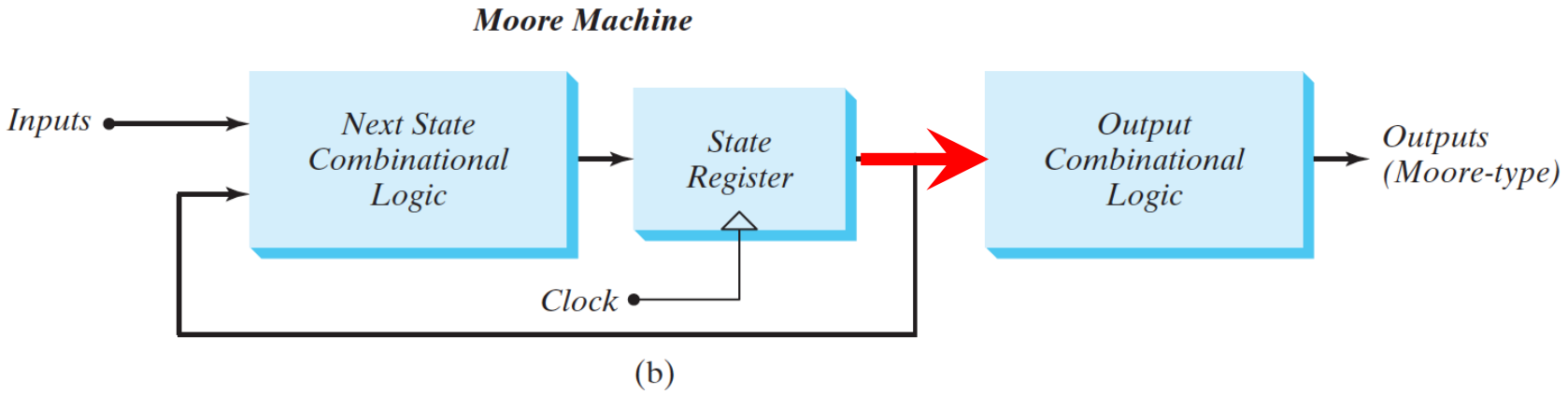


FIGURE 5.21
Block diagrams of Mealy and Moore state machines

Sequential Logic

Flip-Flop + Inputs + Outputs

Mealy Model

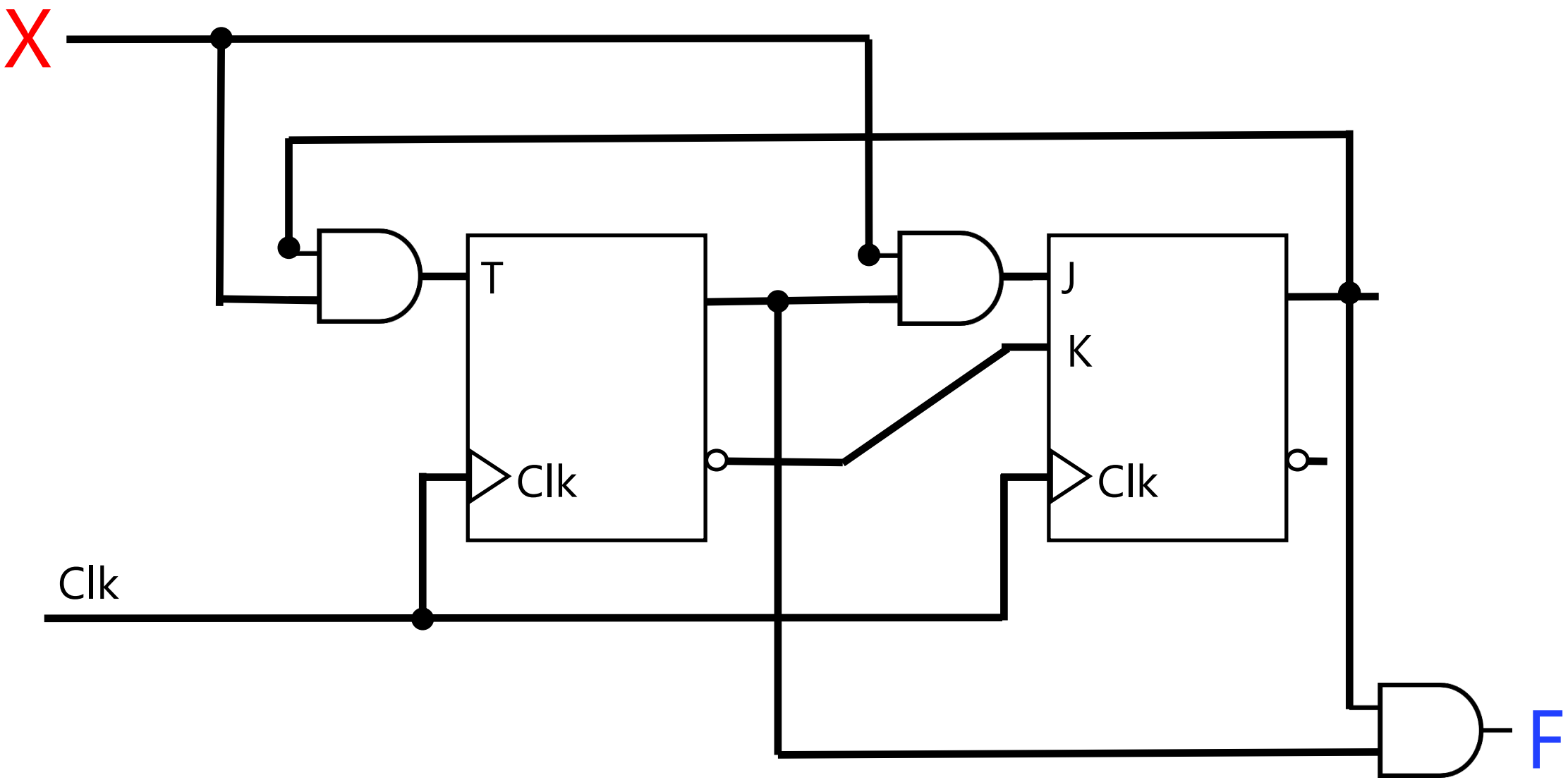
Analysis (Moore model in output)
by an example

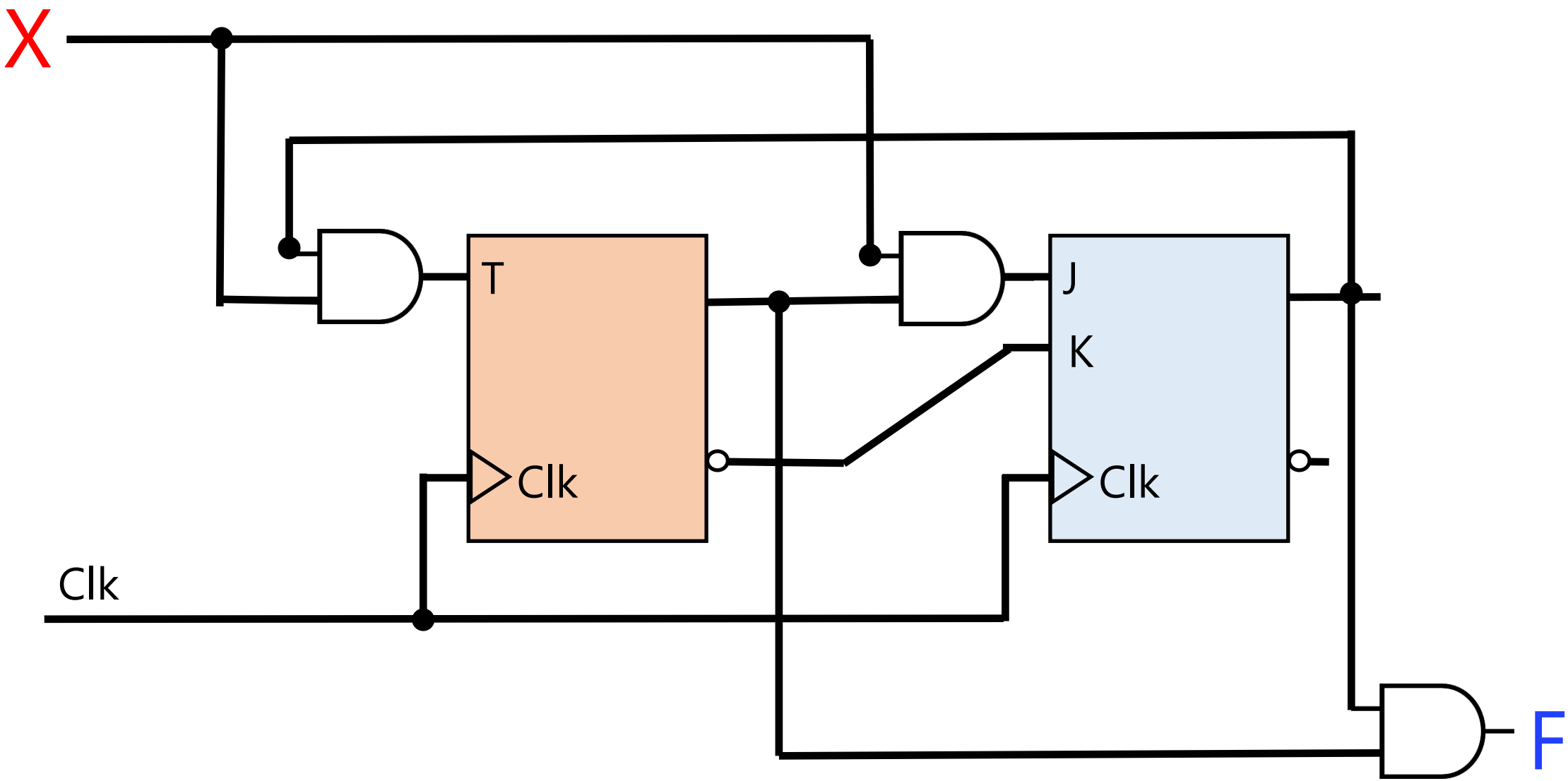
Analysis (Recap)

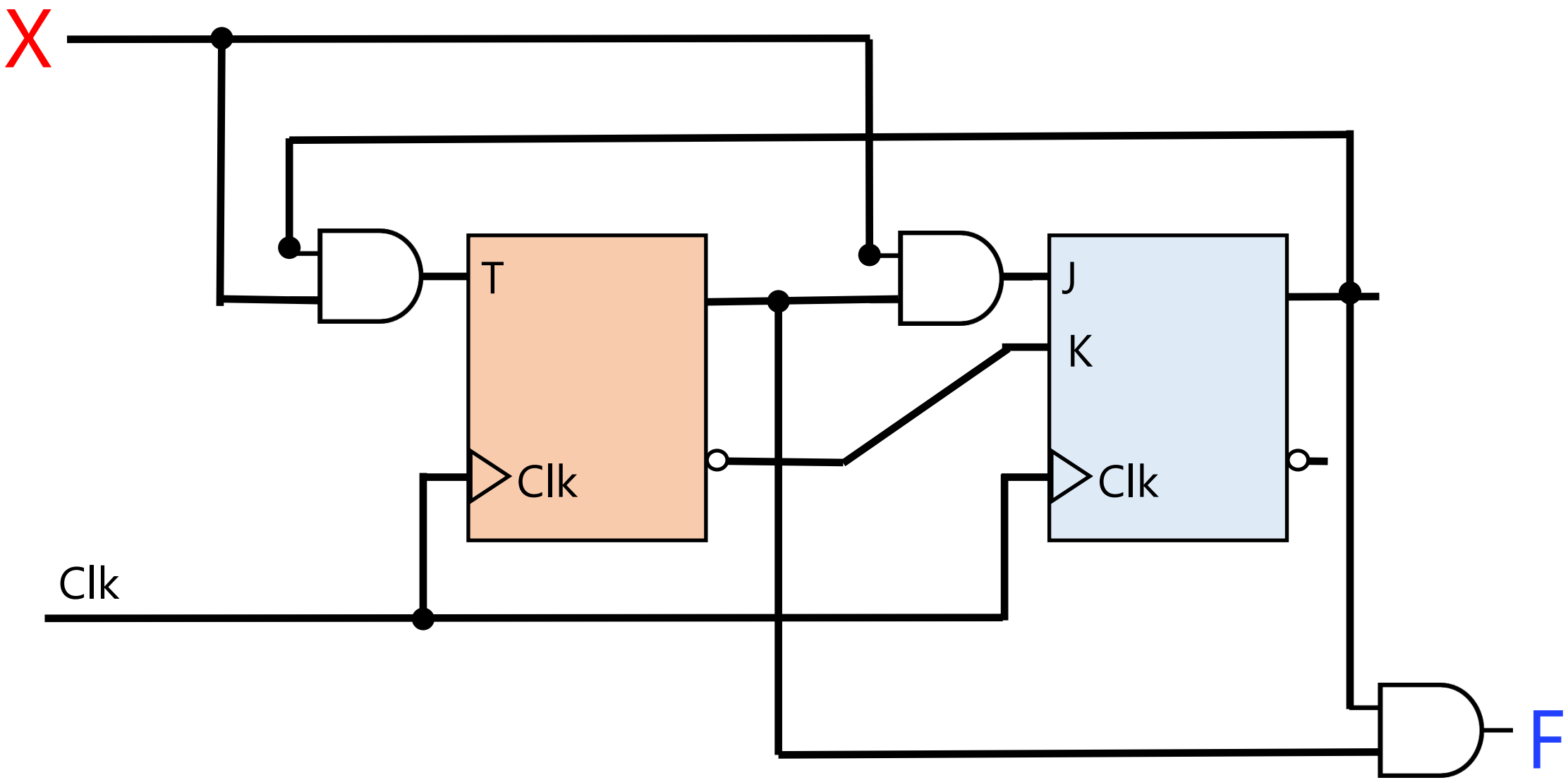
0. Is the circuit sequential or combinational? Any FF or feedback → Sequential
1. What are the flip-flops? RS, D, T, JK, or mixed (e.g., 2 JK, 1 RS, …)
2. What are the state combinations? $2^{\#FF}$
3. Form "State" table:
   a) Columns: for each FF, two columns:
      - one for current state,
      - one for next state
   b) Rows: for each state combination
      - In total: $2^{\#FF}$
4. Fill the state table for next state columns based on:
   a) the current state
   b) the inputs to the FFs
5. Form State Transition Diagram
6. (Optional) Analyze paths and states in state transition diagram

Analysis (+ Input + Moore Model Output)

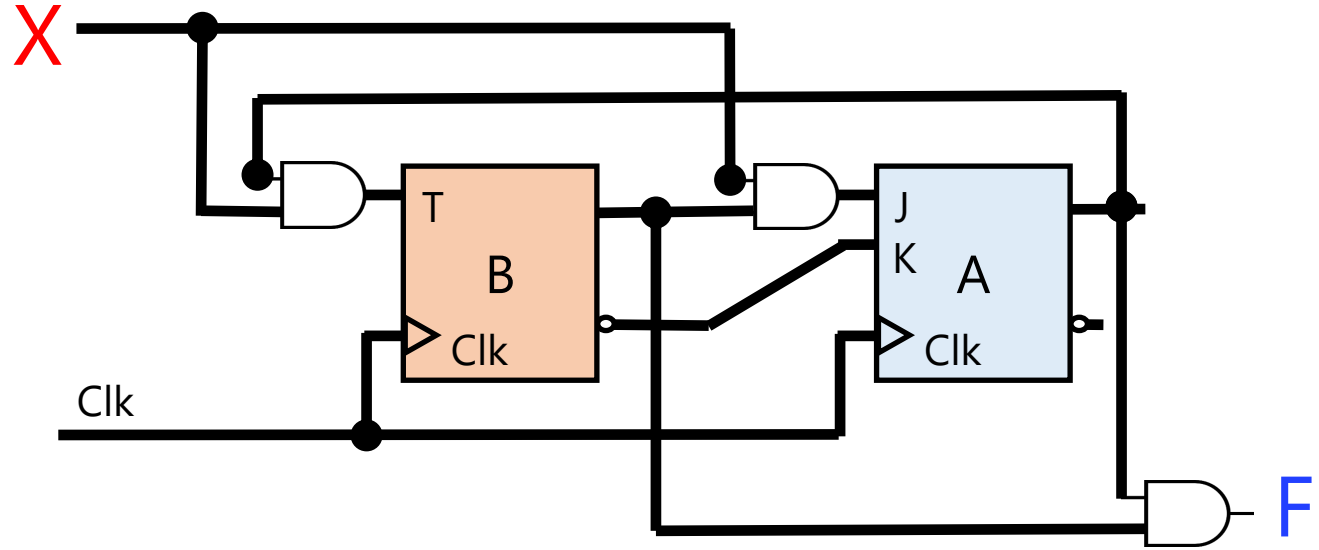
0. Is the circuit sequential or combinational? Any FF or feedback → Sequential
1. What are the flip-flops? RS, D, T, JK, or mixed (e.g., 2 JK, 1 RS, …)
2. What are the state combinations? ~~2^#FF~~
3. Form "State" table:
   a) Columns: for each FF, two columns:
      - one for current state,
      - one for next state
   b) Rows: for each state combination
      - In total: ~~2^#FF~~
4. Fill the state table for next state columns based on:
   a) the current state
   b) the inputs to the FFs
5. Form State Transition Diagram
6. (Optional) Analyze paths and states in state transition diagram


X
T
Clk
J
K
Clk
Clk
F

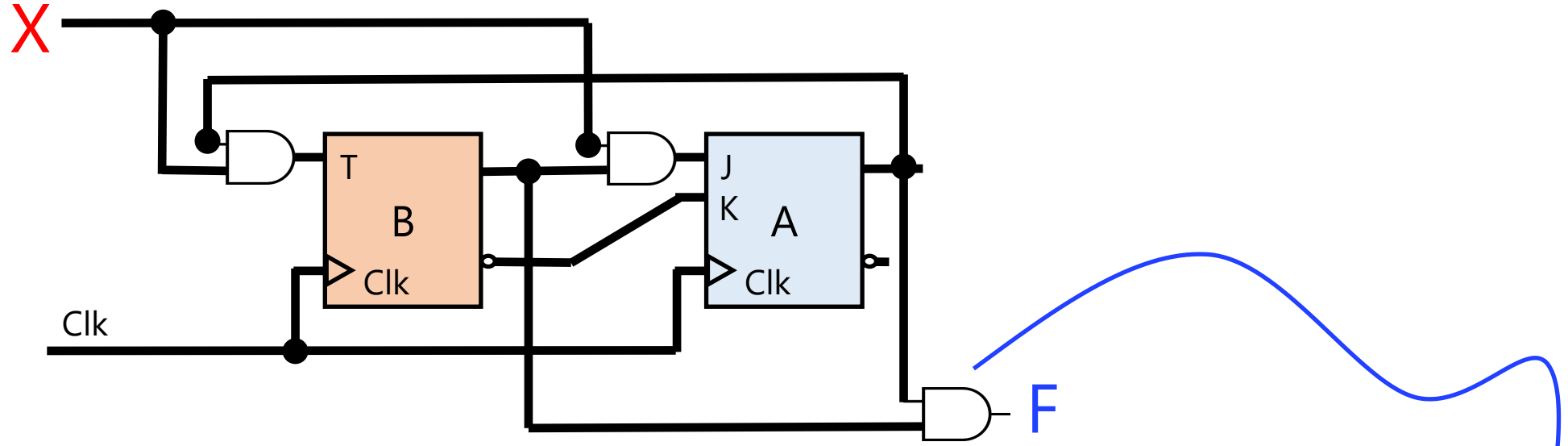

X
T
Clk
J
K
Clk
Clk
F



#FFs + #Inputs = 2+1 → 2^3 = 8 combinations



Inputs	Q(T)		Q(T+1)		Outputs
X	B	A	B	A	F



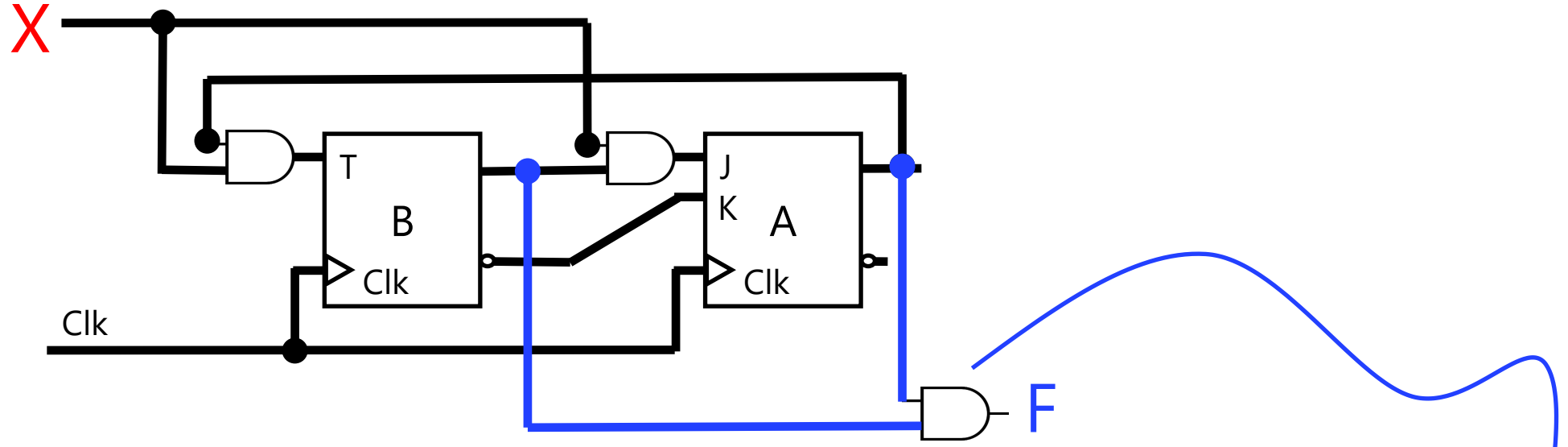
Q(T)		Q(T+1) when X=0		Q(T+1) when X=1		Outputs
B	A	B	A	B	A	F

Moore model:
does not depend on X

Alternative State Table

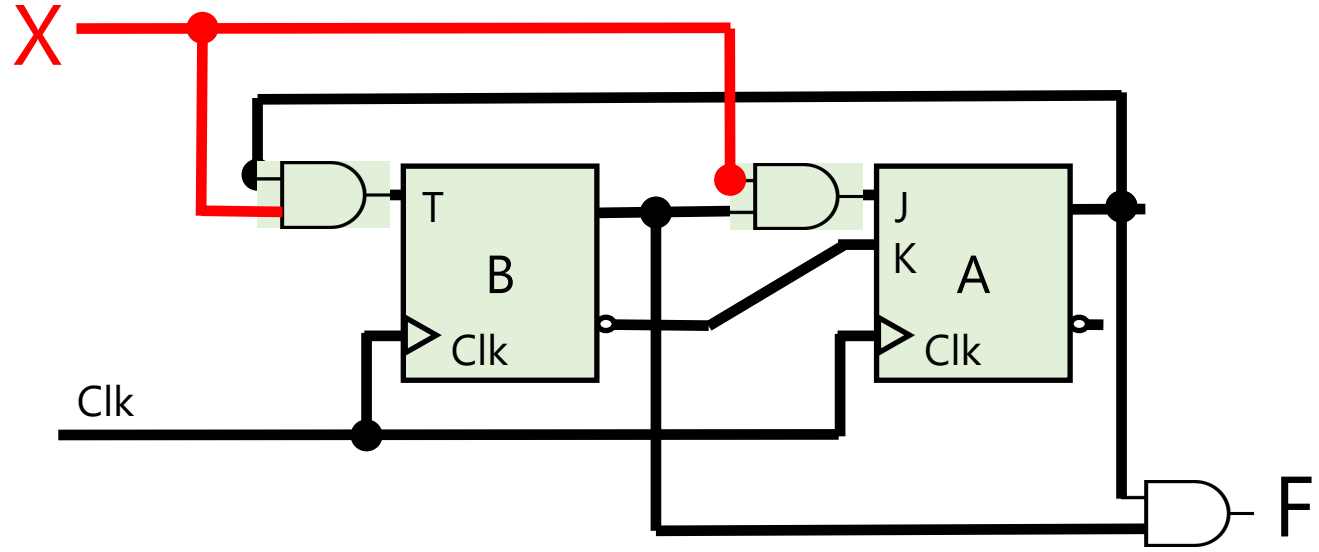
Analysis (Recap)

0. Is the circuit sequential or combinational? Sequential
1. What are the flip-flops? T, JK
2. What are the state combinations? $2^{\#FF} \times 2^{\#inputs} = 2^{\#FF+\#inputs} = 2^3 = 8$
3. Form "State" table:
   a) Columns:
      - For each FF, two columns: one for current state, one for next state
      - For each input, one column
      - For each output, one column
   b) Rows: See item 2
4. Fill the state table for
   a) next state columns
   b) the output value

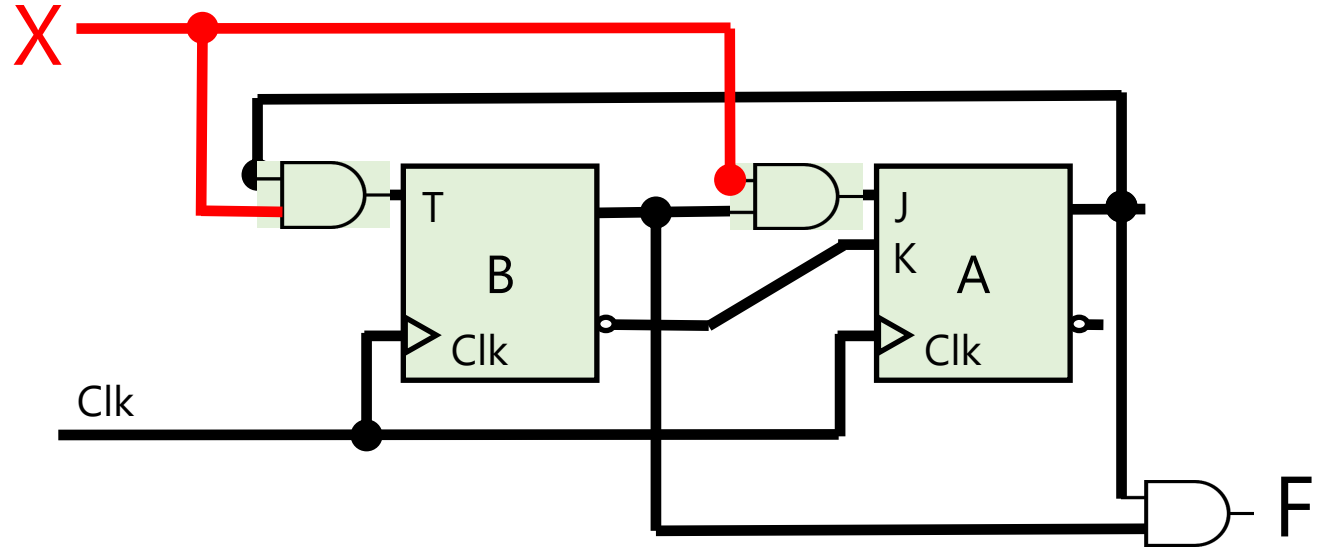


Inputs	Q(T)		Q(T+1)		Outputs
X	B	A	B	A	F=BA
0	0	0			0
0	0	1			0
0	1	0			0
0	1	1			1
1	0	0			0
1	0	1			0
1	1	0			0
1	1	1			1

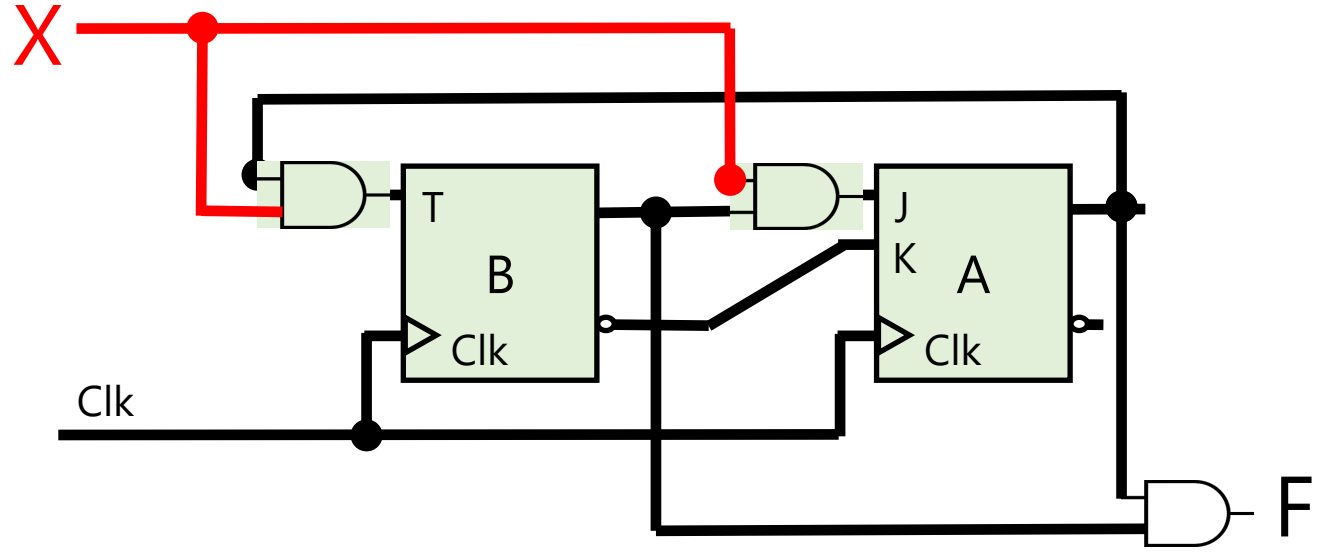
Moore Model
Only depends on current state
X is not involved!



Inputs	Q(T)		Q(T+1)		Outputs
X	B	A	B	A	F=BA
0	0	0		A=0 J_A=XB=00=0 K_A=B′=0′=1 ---------- Reset → 0	0
0	0	1			0
0	1	0			0
0	1	1			1
1	0	0			0



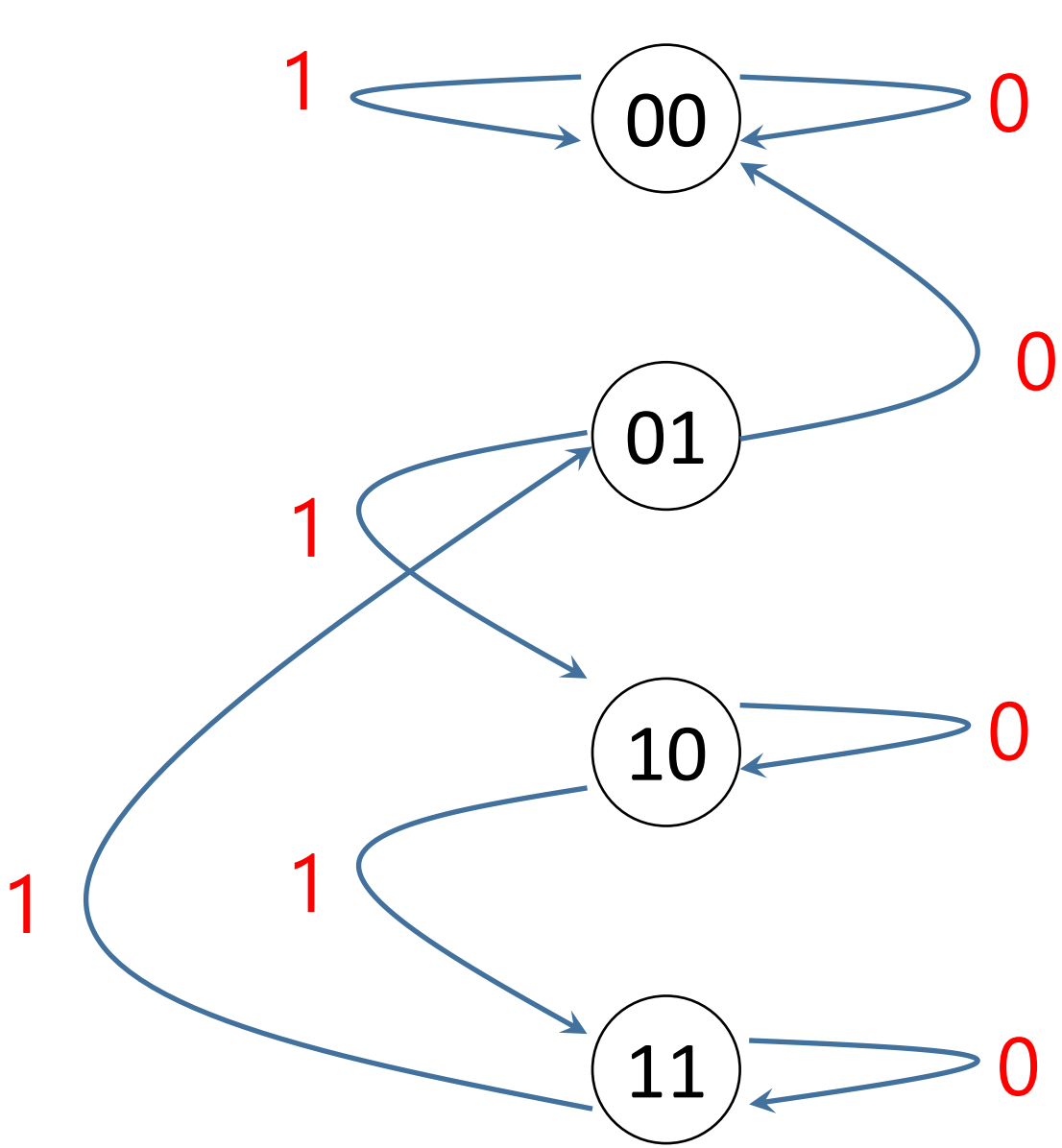
Inputs	Q(T)		Q(T+1)		Outputs
X	B	A	B	A	F=BA
0	0	0	B=0 T_B=XA=00=0 ---------- Store → 0	0	0
0	0	1			0
0	1	0			0
0	1	1			1
1	0	0			0
1	0	1			0



Inputs	Q(T)		Q(T+1)		Outputs
X	B	A	B	A	F=BA
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	1	0	0
1	1	0	1	1	0
1	1	1	0	1	1

Analysis (Recap)

0. Is the circuit sequential or combinational? Sequential
1. What are the flip-flops? T, JK
2. What are the state combinations? $2^{\#FF} \times 2^{\#inputs} = 2^{\#FF+\#inputs} = 2^3 = 8$
3. Form "State" table:
   a) Columns:
      - For each FF, two columns: one for current state, one for next state
      - For each input, one column
      - For each output, one column
   b) Rows: See item 2
4. Fill the state table for
   a) next state columns
   b) the output value
5. Form state (transition) diagram
   a) nodes for states, directed edges for transitions between states
   b) labels for edges by the value of input
   c) labels for nodes by the value of output

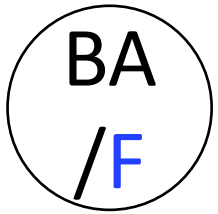


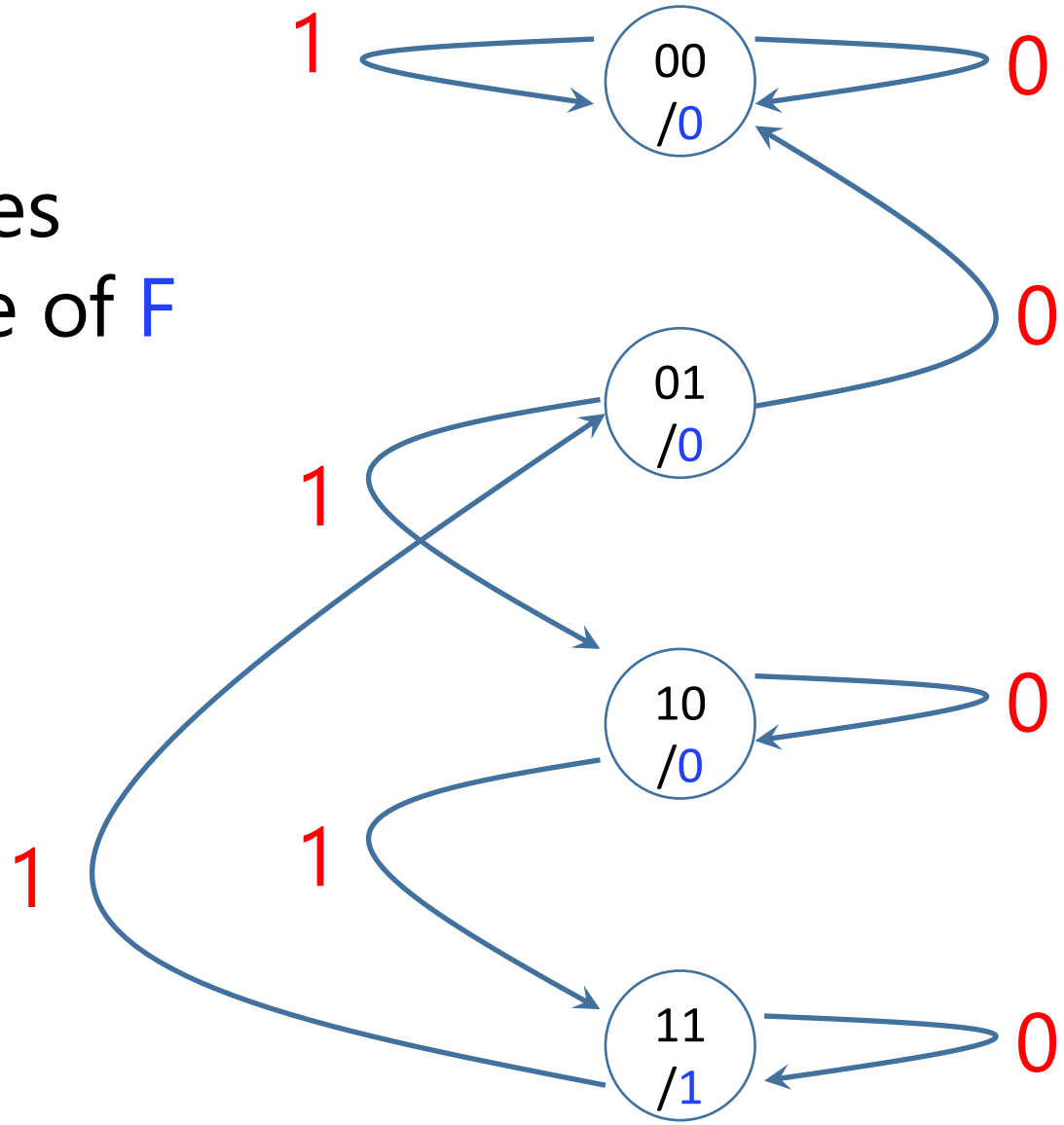
BA

Labels on edges based on value of X

X

Labels on nodes
based on value of F

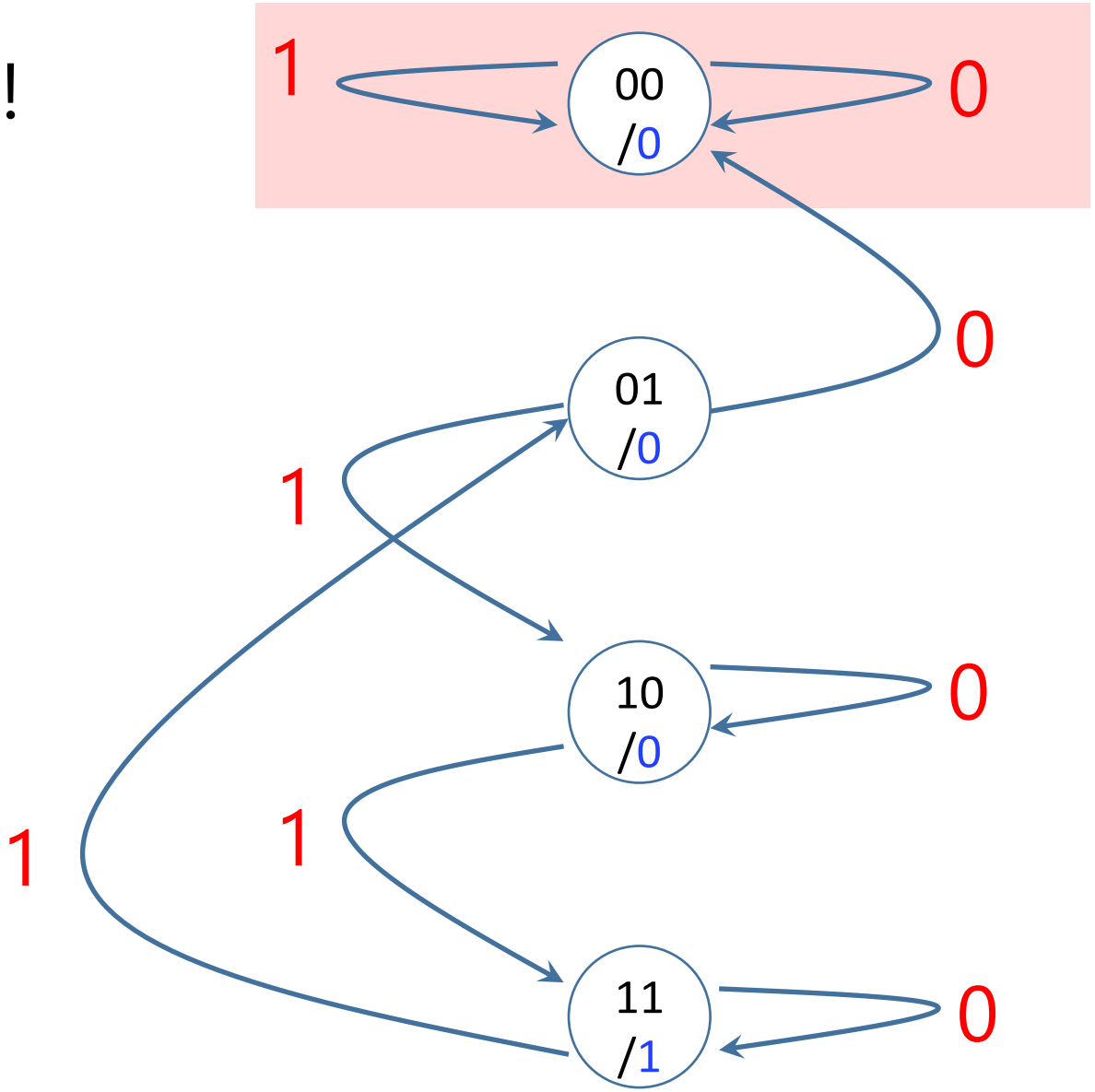




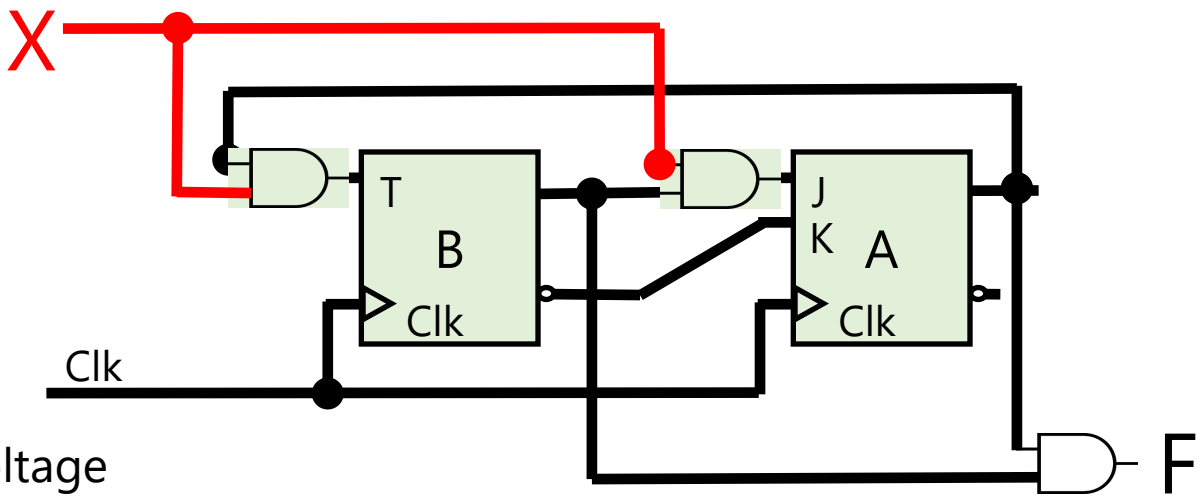
Analysis

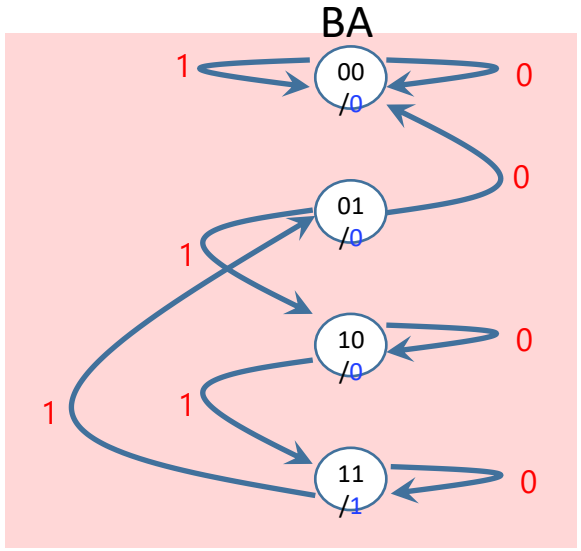
6) (*Optional*) Path on State Transitions

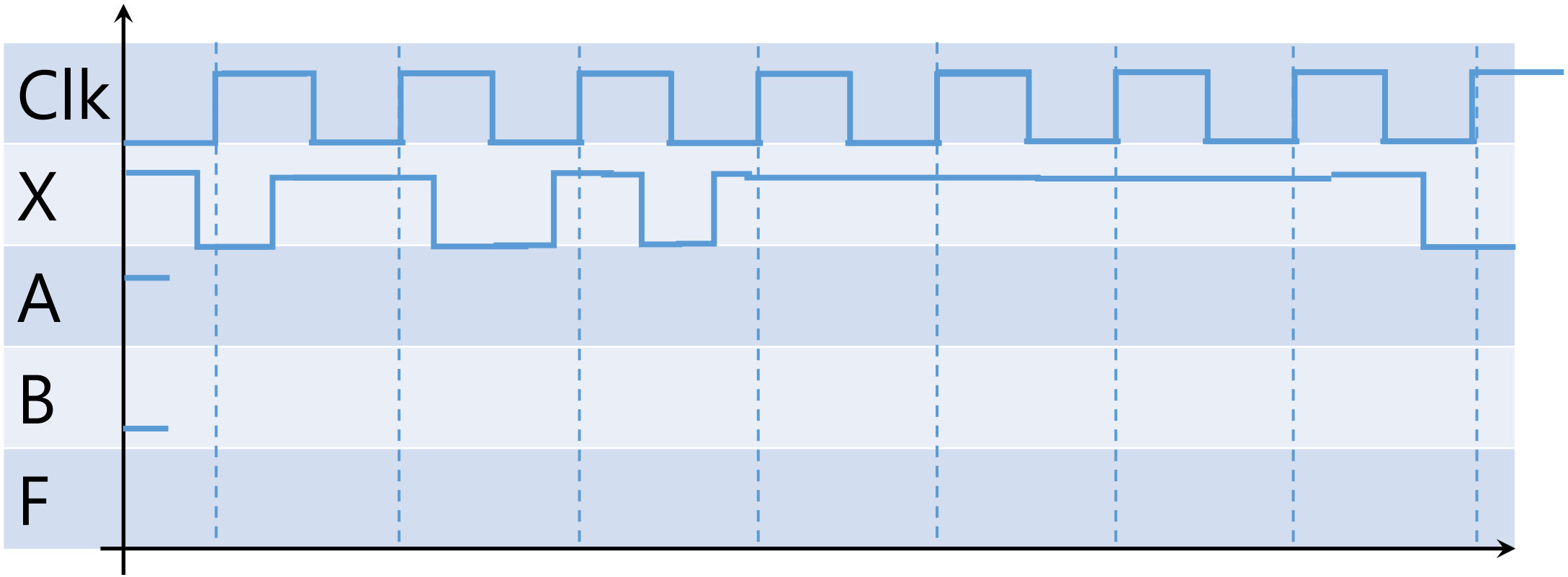
Life lock!



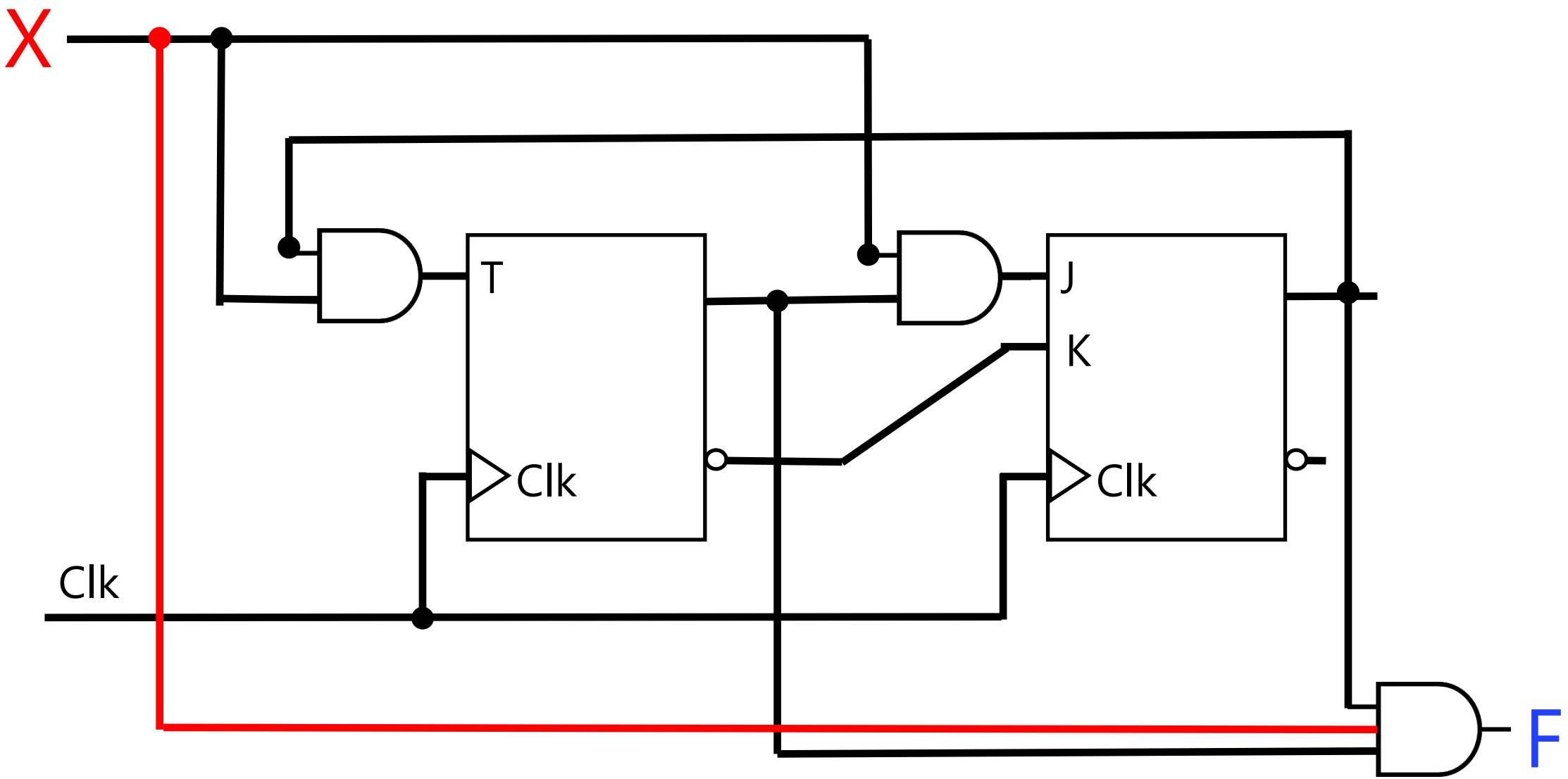
Synchronization
Practice Timing Diagram


X
T
B
Clk
J
K
A
Clk
Clk
F


BA
00
/0
01
/0
10
/0
11
/1
1
0
1
0
1
0
1
0

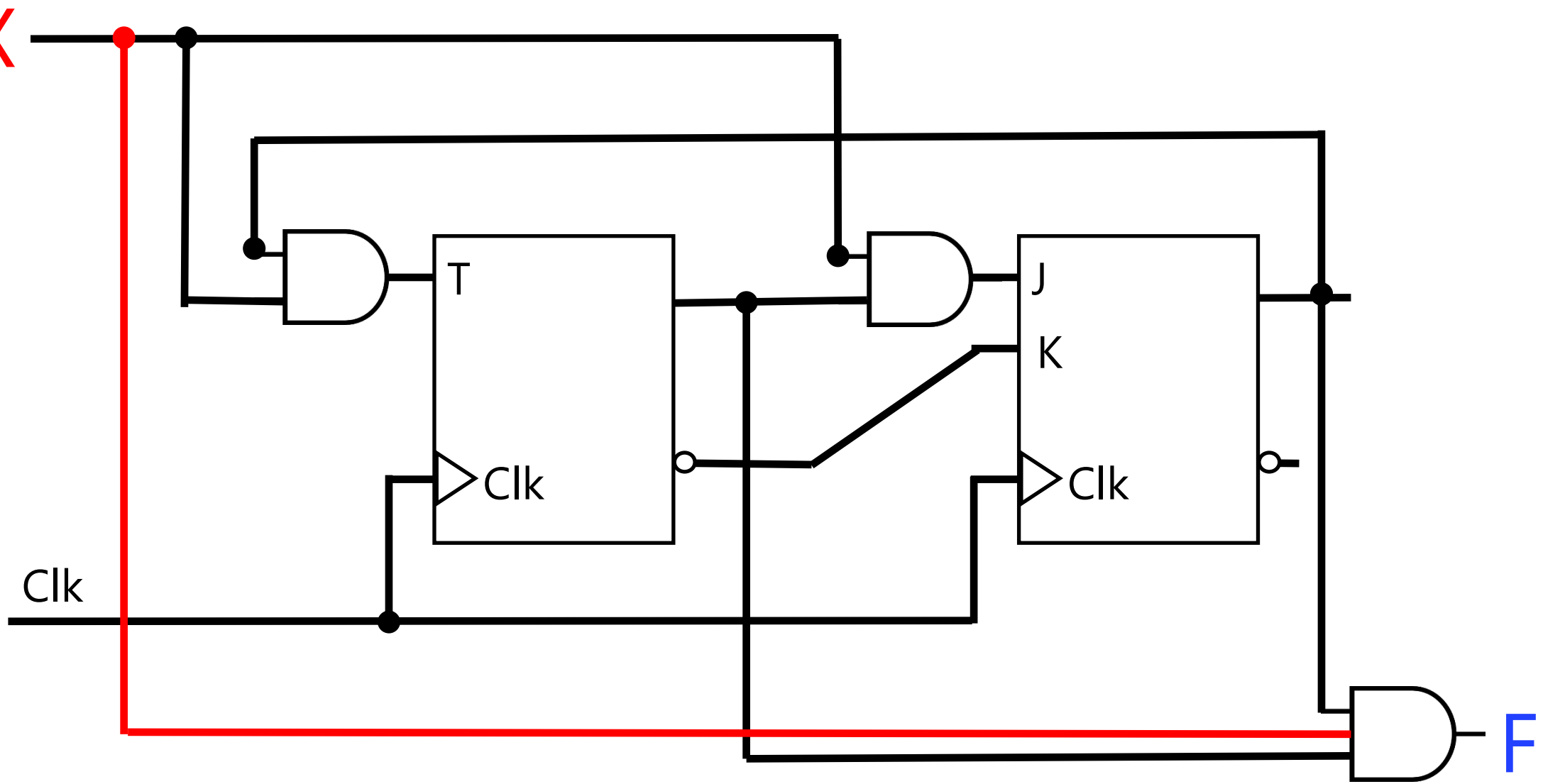

Voltage
Clk
X
A
B
F

Asynchronized



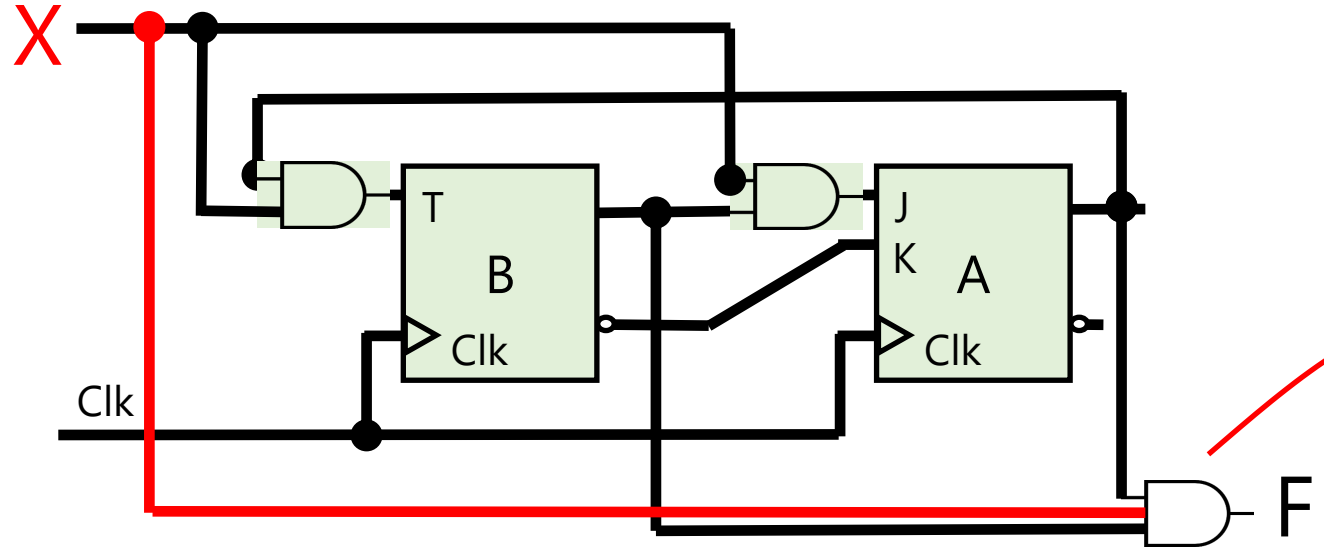
Mealy Model

F depends on X



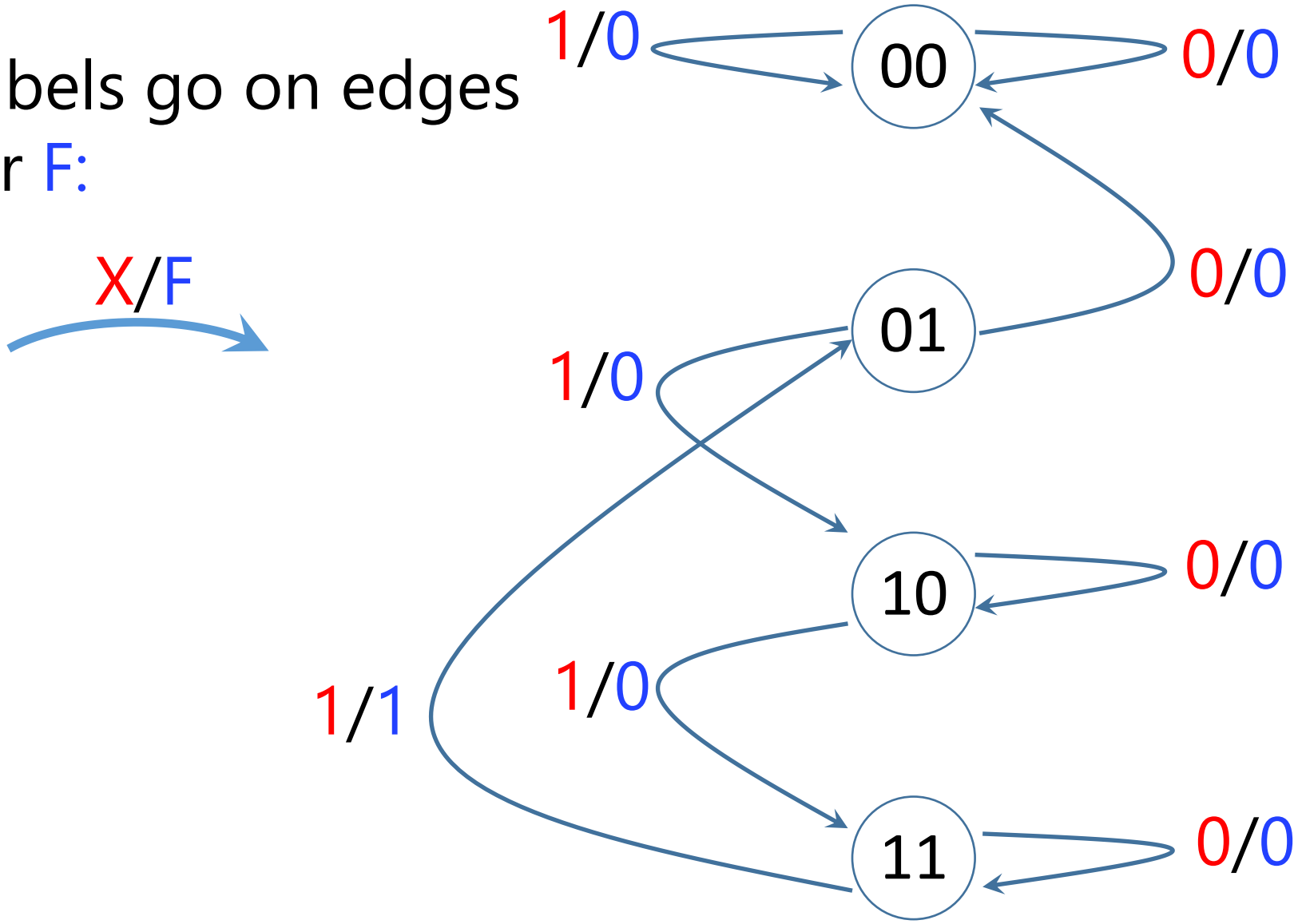
Mealy Model

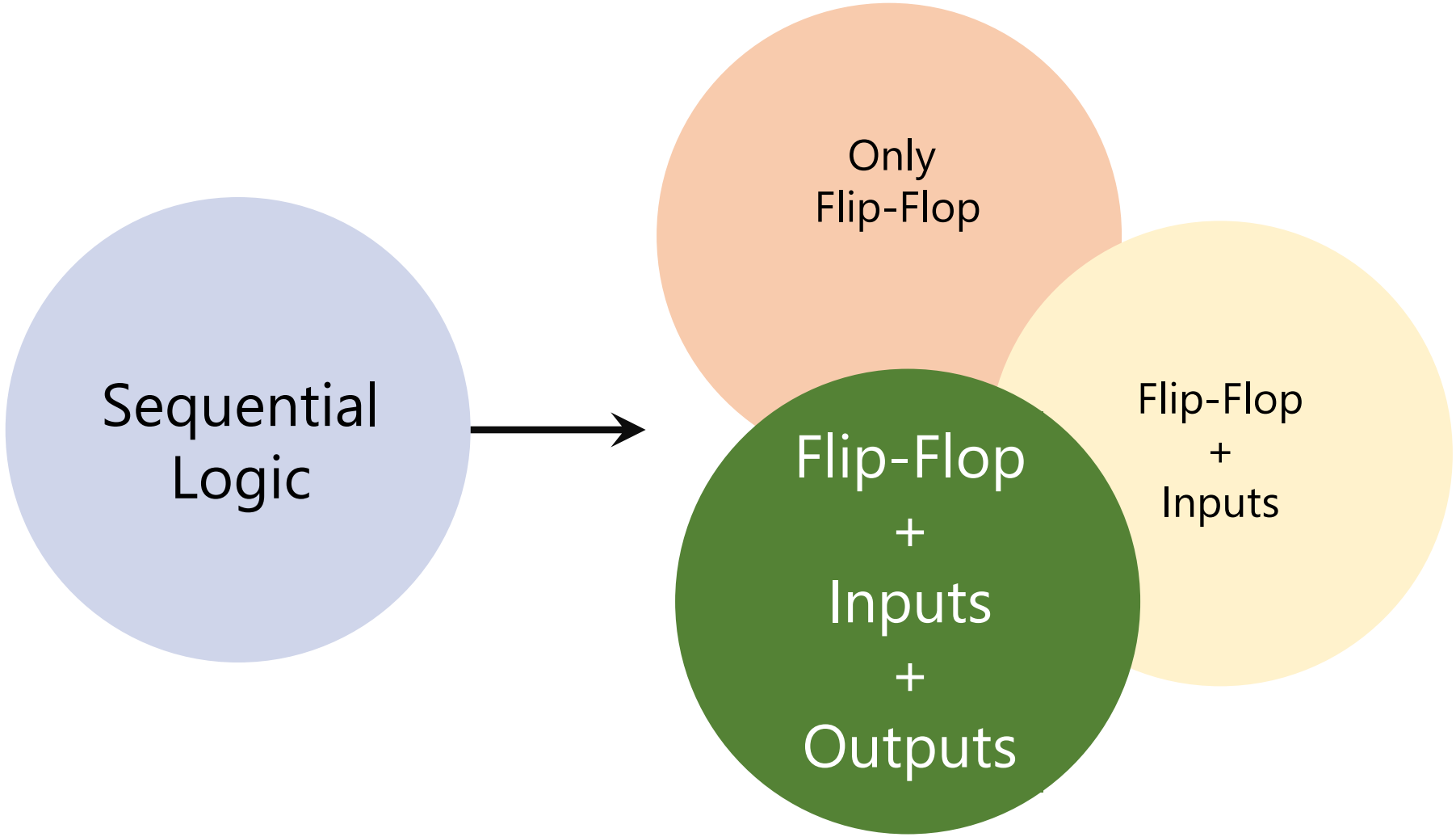
F depends on X, so, F can change out of Clk synchronization!



Inputs	Q(T)		Q(T+1)		Outputs
X	B	A	B	A	F=XBA
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	1	1	0	0
1	1	0	1	1	0
1	1	1	0	1	1

Labels go on edges for F:




Sequential Logic
Only Flip-Flop
Flip-Flop + Inputs
Flip-Flop + Inputs + Outputs

Design
by an example

Counter Up-Down

Switch to count up from i to i+1

Switch to count down from i to i-1

Turn on the light if the current number is even

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?

Counter up/down

0 → 1 → 2 → 3 → … → N-1 → N

N → N-1 → … → 3 → 2 → 1 → 0

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF

Counter up/down

N=7

000→001→010→011→100→101→110→111

000←001←010←011←100←101←110←111

For each intermediate state, we need 3 bits → 3 flip-flops

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?

Counter up/down
N=7

000→001→010→011→100→101→110→111

000←001←010←011←100←101←110←111

We need to switch between up and down → 1 binary variable

X=0 Count Up

X=1 Count Down

Counter up/down

N=7

X=0: 000→001→010→011→100→101→110→111

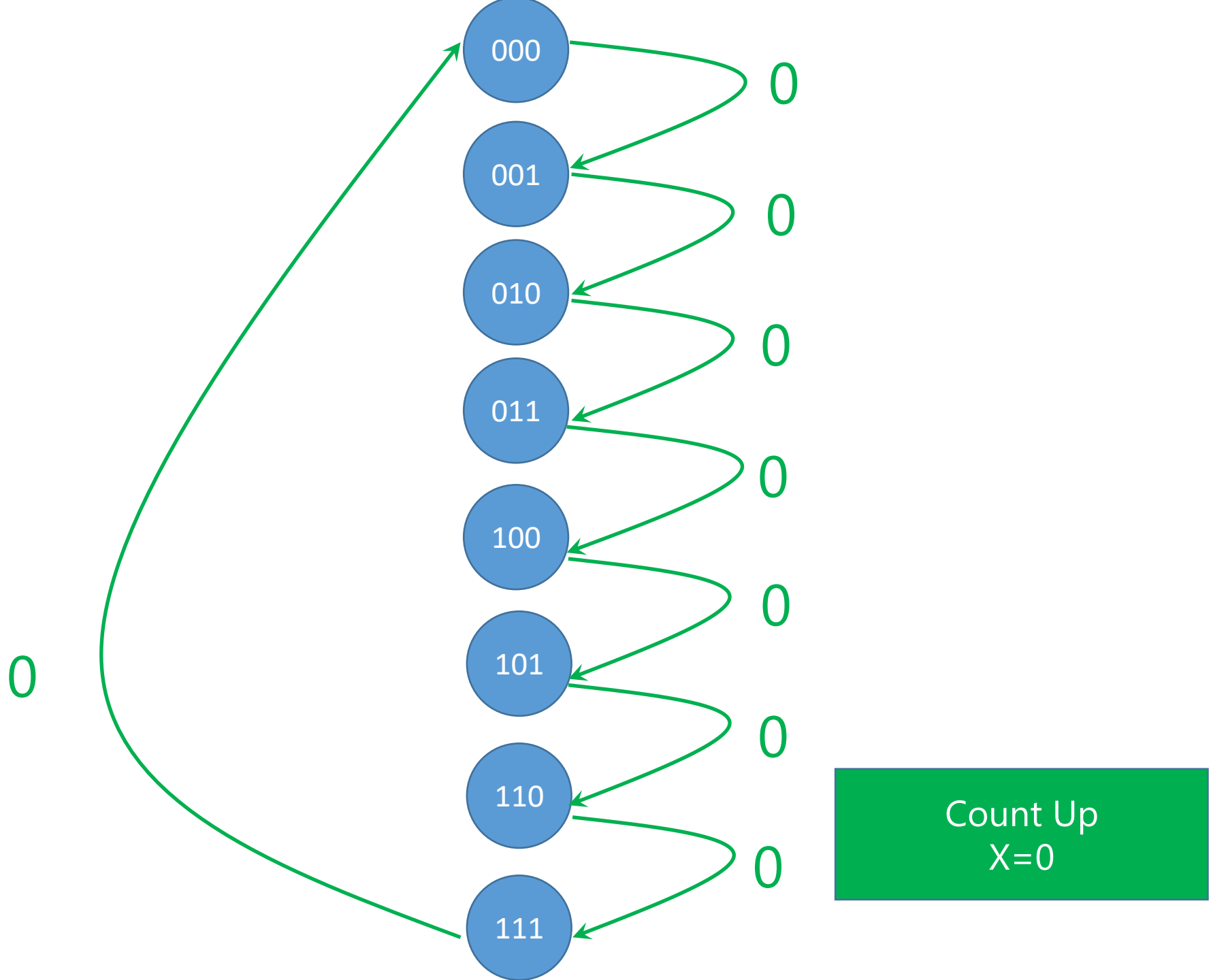
X=1: 111→110→101→100→011→010→001→000

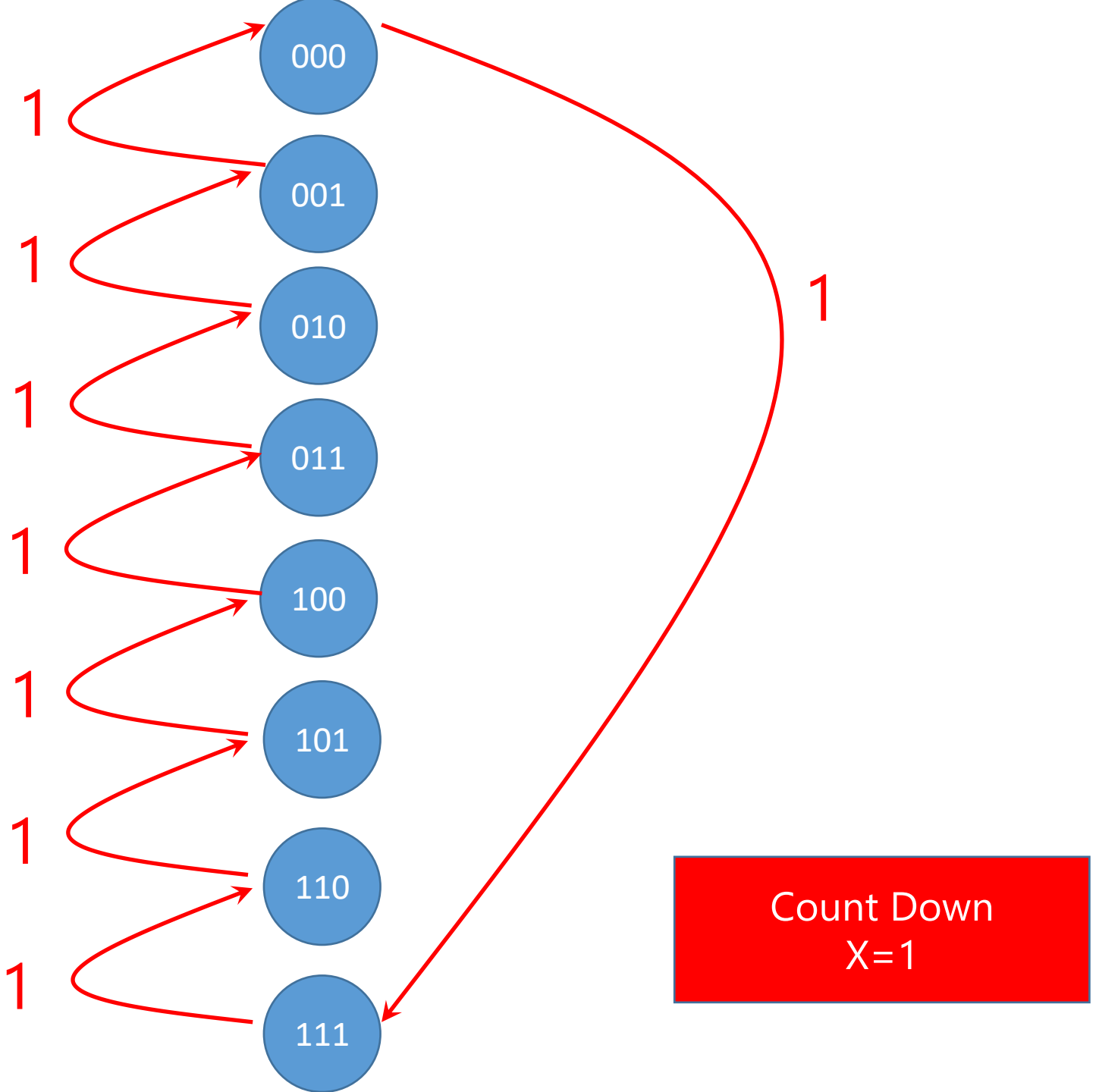
We need to turn on the light when the number is even

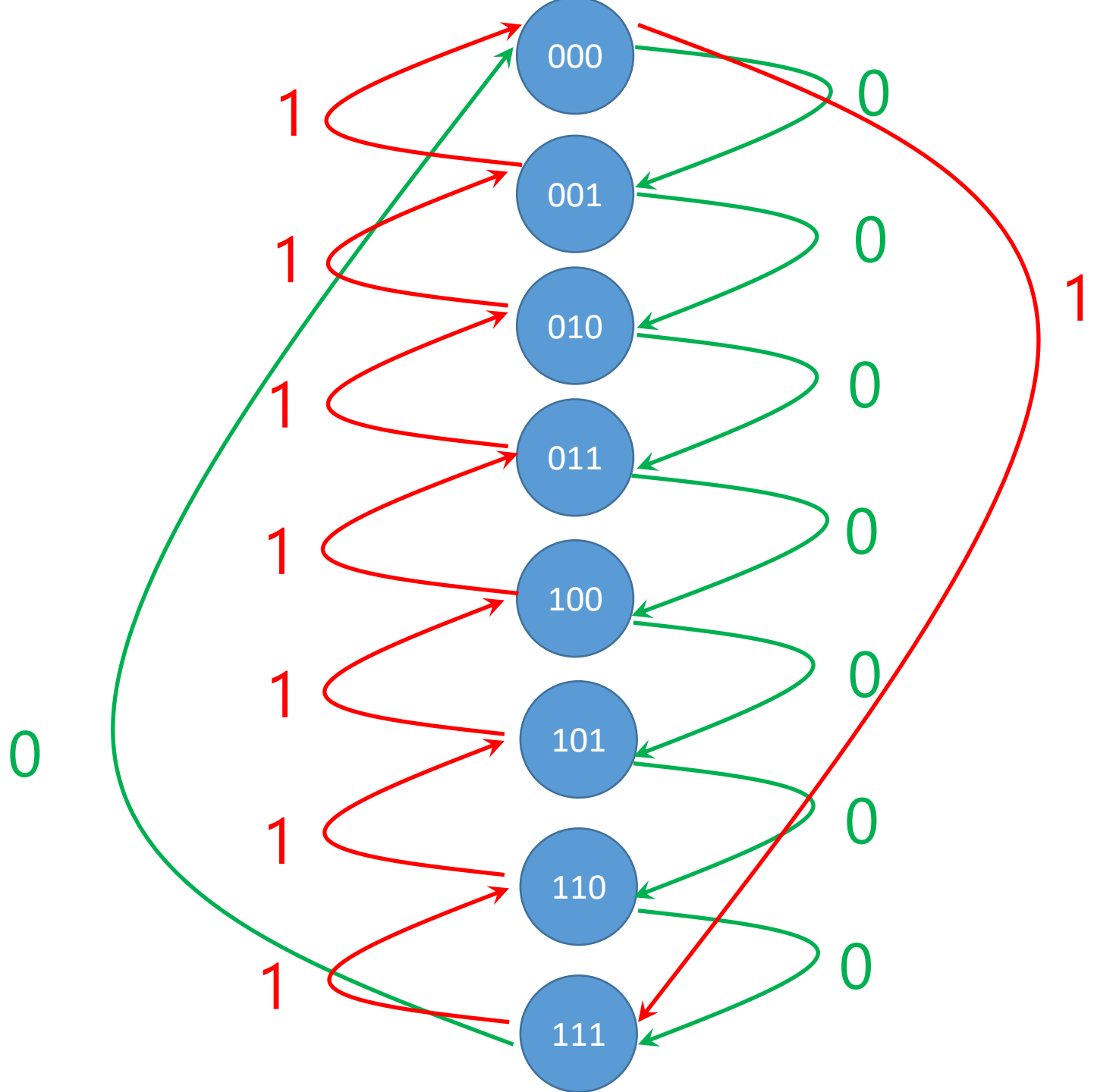
→ 1 binary variable → F

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?
3. Form the state (transition) diagram


000
001
010
011
100
101
110
111
0
0
0
0
0
0
0
0
0
Count Up
X=0

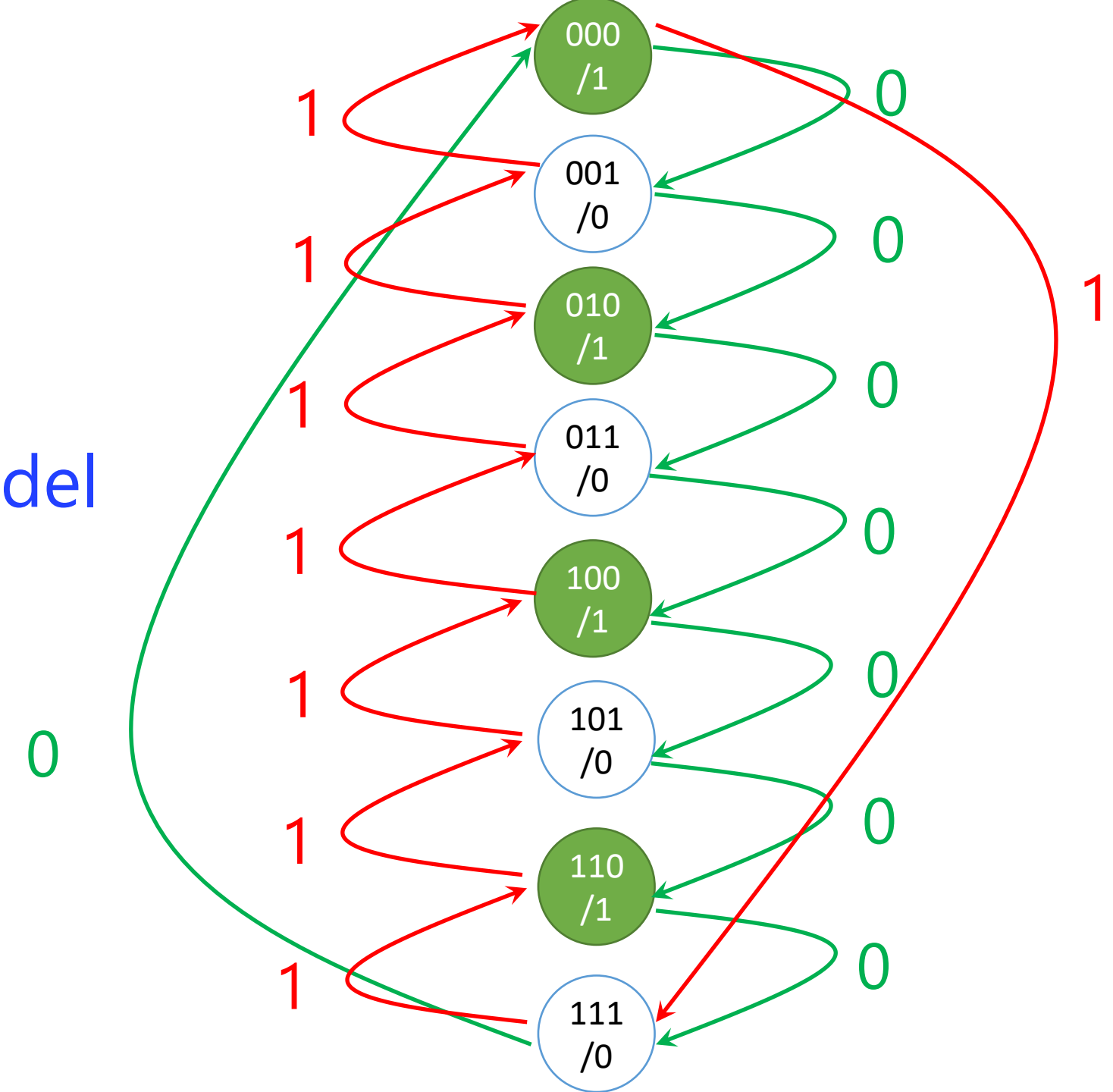

000
001
010
011
100
101
110
111
1
1
1
1
1
1
1
1
Count Down
X=1


000
001
010
011
100
101
110
111
1
0
1
0
1
0
1
0
1
0
1
0
1
0
1
0

F=1 : even

F=0 : else

Moore model



Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?
3. Form the state (transition) diagram
4. Form the state table
5. Fill the state table

Inputs	Q(T)			Q(T+1)			Outputs
X	C	B	A	C	B	A	F
0	0	0	0	0	0	1	
0	0	0	1	0	1	0	
0	0	1	0	0	1	1	
0	0	1	1	1	0	0	
0	1	0	0	1	0	1	
0	1	0	1	1	1	0	
0	1	1	0	1	1	1	
0	1	1	1	0	0	0	
1	0	0	0	1	1	1	
1	0	0	1	0	0	0	
1	0	1	0	0	0	1	
1	0	1	1	0	1	0	
1	1	0	0	0	1	1	
1	1	0	1	1	0	0	
1	1	1	0	1	0	1	
1	1	1	1	1	1	0	

Count Up
X=0

Count Down
X=1

Inputs	Q(T)			Q(T+1)			Outputs
X	C	B	A	C	B	A	F
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	1	1
0	0	1	1	1	0	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	1	0	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	1	1
1	0	1	1	0	1	0	0
1	1	0	0	0	1	1	1
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	1
1	1	1	1	1	1	0	0

F depends only on current state of CBA

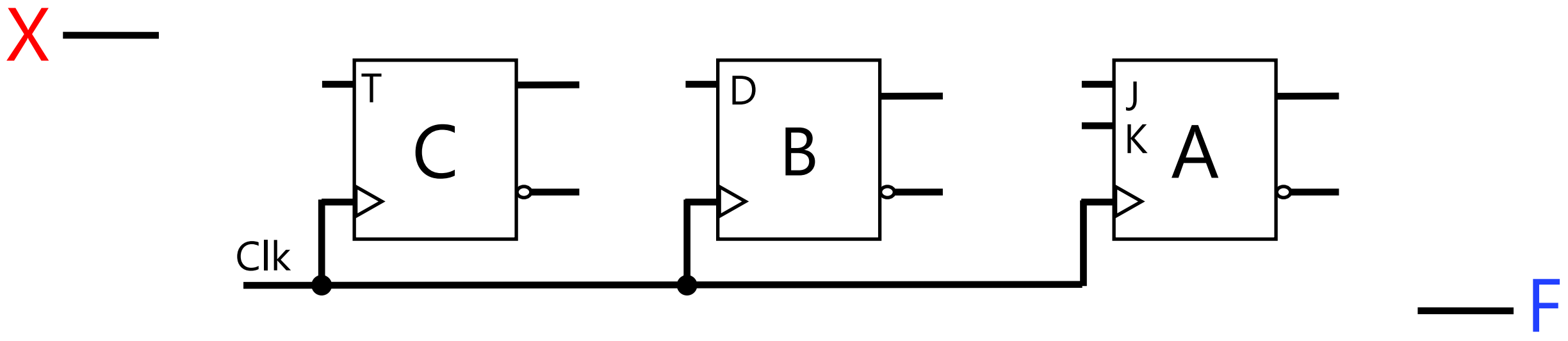
Inputs	Q(T)			Q(T+1)			Outputs
X	C	B	A	C	B	A	F
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	1	1
0	0	1	1	1	0	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	1	1	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	1	1
1	0	1	1	0	1	0	0
1	1	0	0	0	1	1	1
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	1
1	1	1	1	1	1	0	0

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?
3. Form the state (transition) diagram
4. Form the state table
5. Fill the state table
6. What type of storage (flip-flop)? RS, D, T, JK, or Mixed

Counter Up/Down

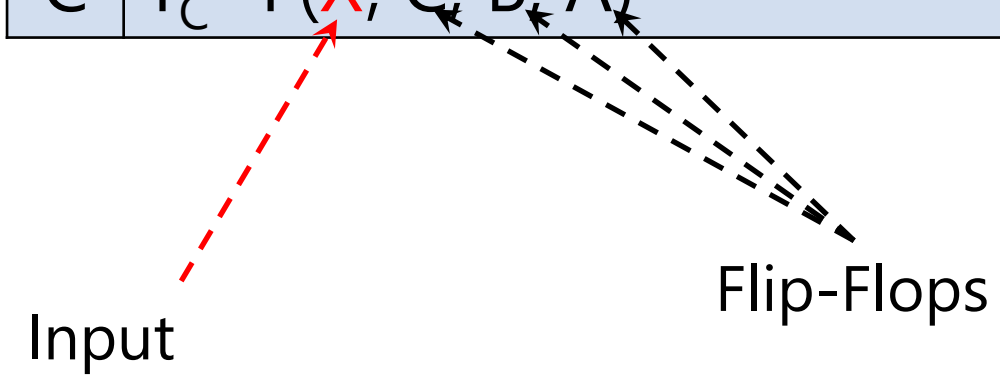
Let’s pick mix FFs: 1×T-FF, 1×D-FF, 1×JK-FF


X
T
C
D
B
J
K
A
Clk
F

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?
3. Form the state (transition) diagram
4. Form the state table
5. Fill the state table
6. What type of storage (flip-flop)? RS, D, T, JK, or Mixed
7. Input (*excitation*) equations for each FF
8. Minimization of input (*excitation*) equations

Excitation Equations		
A	J_A=F(X, C, B, A)	K_A=F(X, C, B, A)
B	D_B=F(X, C, B, A)	
C	T_C=F(X, C, B, A)	



Excitation Equations		
A	J_A=F(X, C, B, A)	K_A=F(X, C, B, A)
B	D_B=	
C	T_C=	

Inputs	Q(T)			Q(T+1)		Outputs	Excitation for A		
X	C	B	A	C	B	A	F	Action	J_A	K_A
0	0	0	0	0	0	1	1	Set/Cmp	1	X
0	0	0	1	0	1	0	0	Rst/Cmp	X	1
0	0	1	0	0	1	1	1	Set/Cmp	1	X
0	0	1	1	1	0	0	0	Rst/Cmp	X	1
0	1	0	0	1	0	1	1	Set/Cmp	1	X
0	1	0	1	1	1	0	0	Rst/Cmp	X	1
0	1	1	0	1	1	1	1	Set/Cmp	1	X
0	1	1	1	0	0	0	0	Rst/Cmp	X	1
1	0	0	0	1	1	1	1	Set/Cmp	1	X
1	0	0	1	0	0	0	0	Rst/Cmp	X	1
1	0	1	0	0	0	1	1	Set/Cmp	1	X
1	0	1	1	0	1	0	0	Rst/Cmp	X	1
1	1	0	0	0	1	1	1	Set/Cmp	1	X
1	1	0	1	1	0	0	0	Rst/Cmp	X	1
1	1	1	0	1	0	1	1	Set/Cmp	1	X
1	1	1	1	1	1	0	0	Rst/Cmp	X	1

Excitation Equations		
A	$J_A=\sum(0,2,4,6,8,10,12,14)+d(1,3,5,7,9,11,13,15)$	$K_A=\sum(1,3,5,7,9,11,13,15)+d(0,2,4,6,8,10,12,14)$
B	$D_B=$	
C	$T_C=$	

Inputs	Q(T)				Q(T+1)		Outputs		Excitation for B	
X	C	B	A	C	B	A	F	Action	D_B
0	0	0	0	0	0	1	1	Rst	0
0	0	0	1	0	1	0	0	Set	1
0	0	1	0	0	1	1	1	Set	1
0	0	1	1	1	0	0	0	Rst	0
0	1	0	0	1	0	1	1	Rst	0
0	1	0	1	1	1	0	0	Set	1
0	1	1	0	1	1	1	1	Set	1
0	1	1	1	0	0	0	0	Rst	0
1	0	0	0	1	1	1	1	Set	1
1	0	0	1	0	0	0	0	Rst	0
1	0	1	0	0	0	1	1	Rst	0
1	0	1	1	0	1	0	0	Set	1
1	1	0	0	0	1	1	1	Set	1
1	1	0	1	1	0	0	0	Rst	0
1	1	1	0	1	0	1	1	Rst	0
1	1	1	1	1	1	0	0	Set	1

Excitation Equations		
A	$J_A=\sum(0,2,4,6,8,10,12,14)+d(1,3,5,7,9,11,13,15)$	$K_A=\sum(1,3,5,7,9,11,13,15)+d(0,2,4,6,8,10,12,14)$
B	$D_B=\sum(1,2,5,6,8,11,12,15)$ *Never "don't care condition" happens in D-FF*	
C	$T_C=$	

Inputs	Q(T)			Q(T+1)			Outputs	Excitation for C	
X	C	B	A	C	B	A	F	Action	T_C
0	0	0	0	0	0	1	1	Store	0
0	0	0	1	0	1	0	0	Store	0
0	0	1	0	0	1	1	1	Store	0
0	0	1	1	1	0	0	0	Comp	1
0	1	0	0	1	0	1	1	Store	0
0	1	0	1	1	1	0	0	Store	0
0	1	1	0	1	1	1	1	Store	0
0	1	1	1	0	0	0	0	Comp	1
1	0	0	0	1	1	1	1	Comp	1
1	0	0	1	0	0	0	0	Store	0
1	0	1	0	0	0	1	1	Store	0
1	0	1	1	0	1	0	0	Store	0
1	1	0	0	0	1	1	1	Comp	1
1	1	0	1	1	0	0	0	Store	0
1	1	1	0	1	0	1	1	Store	0
1	1	1	1	1	1	0	0	Store	0

Excitation Equations		
A	$J_A=\sum(0,2,4,6,8,10,12,14)+d(1,3,5,7,9,11,13,15)$	$K_A=\sum(1,3,5,7,9,11,13,15)+d(0,2,4,6,8,10,12,14)$
B	$D_B=\sum(1,2,5,6,8,11,12,15)$ *Never "don't care condition" happens in D-FF*	
C	$T_C=\sum(3,7,8,12)$ *Never "don't care condition" happens in T-FF*	

Minimization for excitation equations

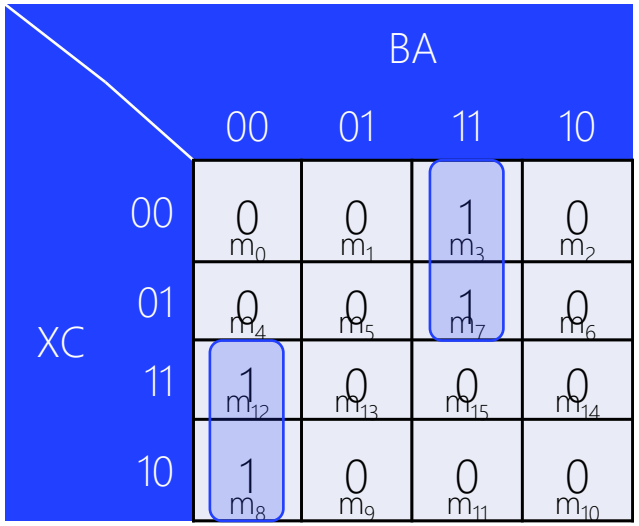
?-Variable K-Map

Excitation Equations		
A	$J_A=\sum(0,2,4,6,8,10,12,14)+d(1,3,5,7,9,11,13,15)$	$K_A=\sum(1,3,5,7,9,11,13,15)+d(0,2,4,6,8,10,12,14)$
B	$D_B=\sum(1,2,5,6,8,11,12,15)$ *Never "don't care condition" happens in D-FF*	
C	$T_C=\sum(3,7,8,12)$ *Never "don't care condition" happens in T-FF*	

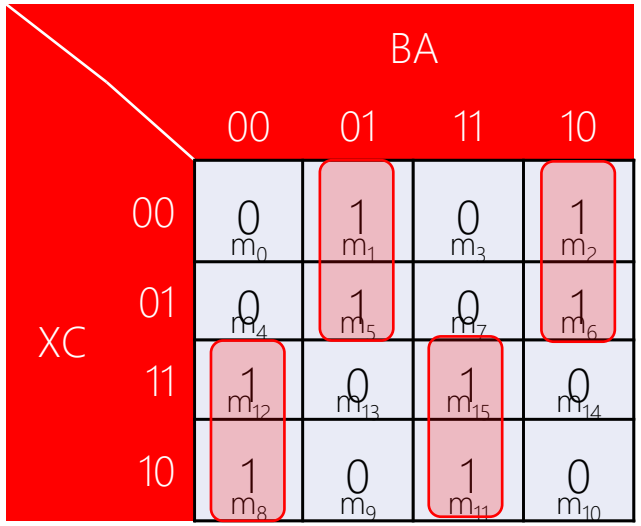
Minimization

4-Variable K-Map

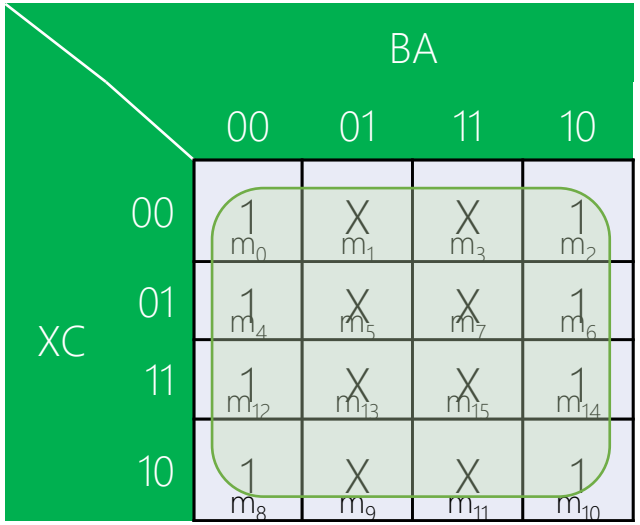
F(X, C, B, A)



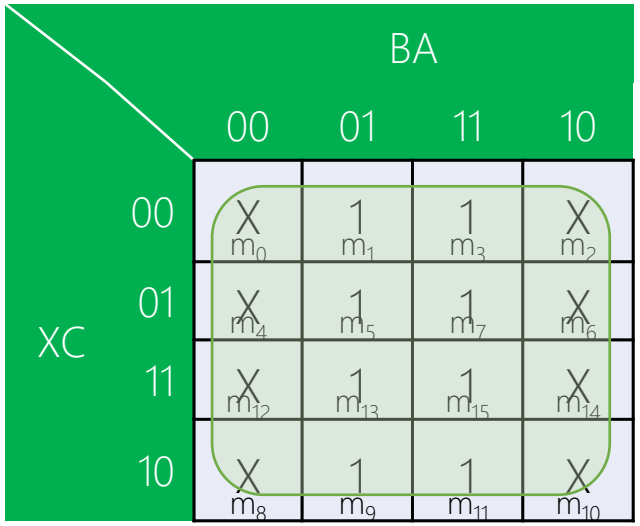
$T_C = \sum(3,7,8,12)$
$= X'BA + XB'A'$



$D_B = \sum(1,2,5,6,8,11,12,15)$
$= X'B'A + X'BA' + XB'A' + XBA$
$= X'(B'A+BA') + X(B'A'+BA)$
$= X'(B \oplus A)+X(B \odot A)$
$= X \oplus B \oplus A$



$J_A = \sum(0,2,4,6,8,10,12,14) +$
$d(1,3,5,7,9,11,13,15) = 1$



$K_A = \sum(1,3,5,7,9,11,13,15) +$
$d(0,2,4,6,8,10,12,14) = 1$

Excitation Equations		
A	$J_A=1$	$K_A=1$
B	$D_B=F(X,C,B,A)=X \oplus B \oplus A$	
C	$T_C=F(X,C,B,A) = X'BA + XB'A'$	

Design (Recap)

0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?
3. Form the state (transition) diagram
4. Form the state table
5. Fill the state table
6. What type of storage (flip-flop)? RS, D, T, JK, or Mixed
7. Input (*excitation*) equations for each FF
8. Minimization of input (*excitation*) equations
9. Boolean function for the output
10. Minimization of output variable

Inputs	Q(T)			Q(T+1)			Outputs
X	C	B	A	C	B	A	F
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	1	1
0	0	1	1	1	0	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	1	1	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	1	1
1	0	1	1	0	1	0	0
1	1	0	0	0	1	1	1
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	1
1	1	1	1	1	1	0	0

Inputs	Q(T)			Q(T+1)			Outputs
X	C	B	A	C	B	A	Y
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	1	1
0	0	1	1	1	0	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	1	1	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	1	1
1	0	1	1	0	1	0	0
1	1	0	0	0	1	1	1
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	1
1	1	1	1	1	1	0	0

$$F = F(C,B,A) = \sum(0,2,4,6)$$

Inputs	Q(T)			Q(T+1)			Outputs
X	C	B	A	C	B	A	Y
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	1	1
0	0	1	1	1	0	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	1	1	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	1	1
1	0	1	1	0	1	0	0
1	1	0	0	0	1	1	1
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	1
1	1	1	1	1	1	0	0

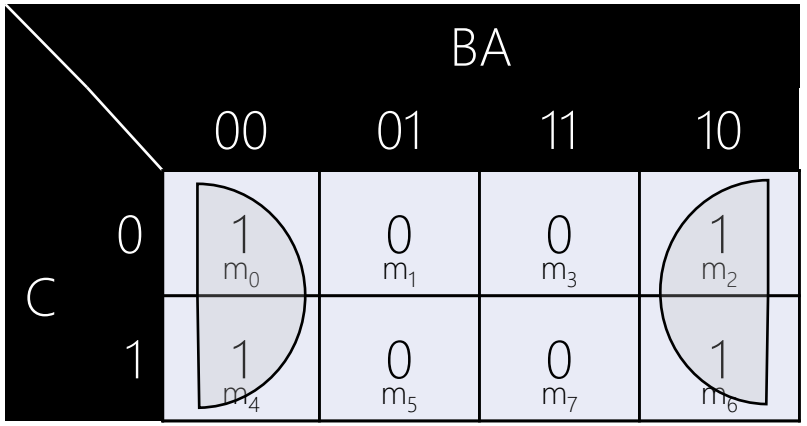
~~Incorrect but if we make a mistake that X is involved:~~

~~$F = F(X,C,B,A) = \sum(0,2,4,6,8,10,12,14)$~~

Minimization for output Y

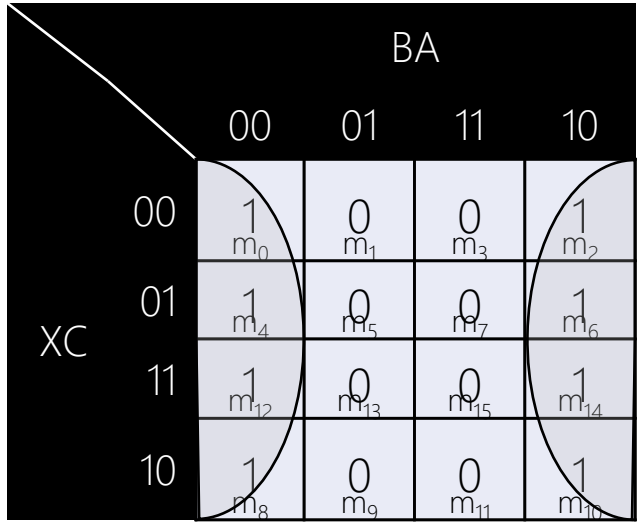
3-Variable K-Map

~~Incorrectly 4-Variable K-Map~~



$F = F(C,B,A) = \sum(0,2,4,6)$
$= A'$

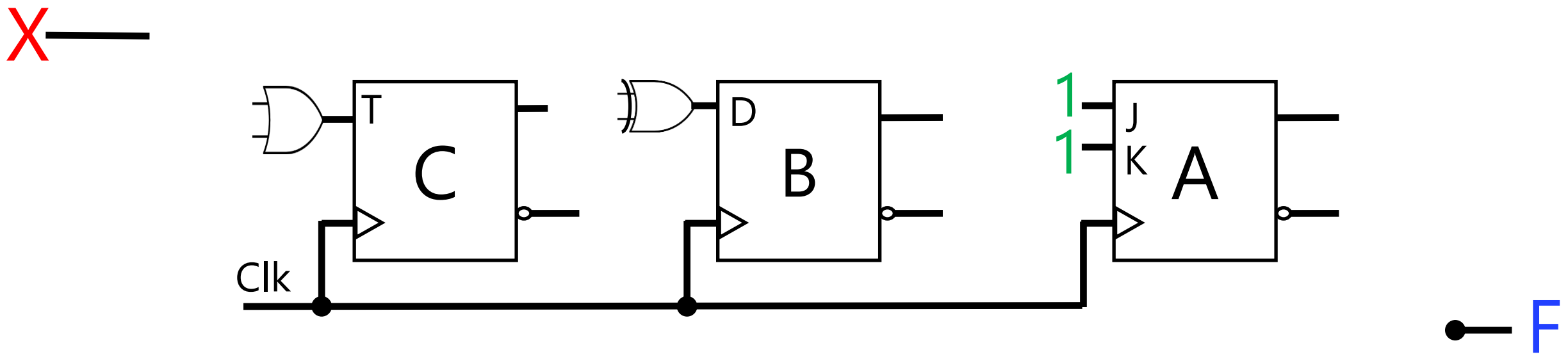
Even if we consider the input, because Y does not depend on X in the state table, it will disappear in simplification!



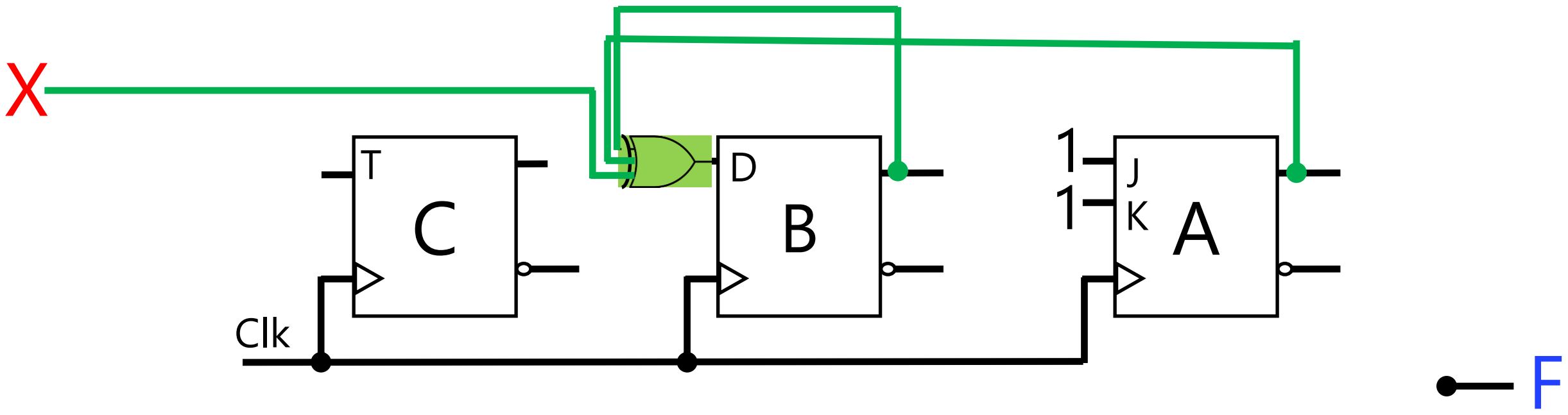
$F = F(X,C,B,A) = \sum(0,2,4,6,8,10,12,14)$
$= A'$

Design (Recap)

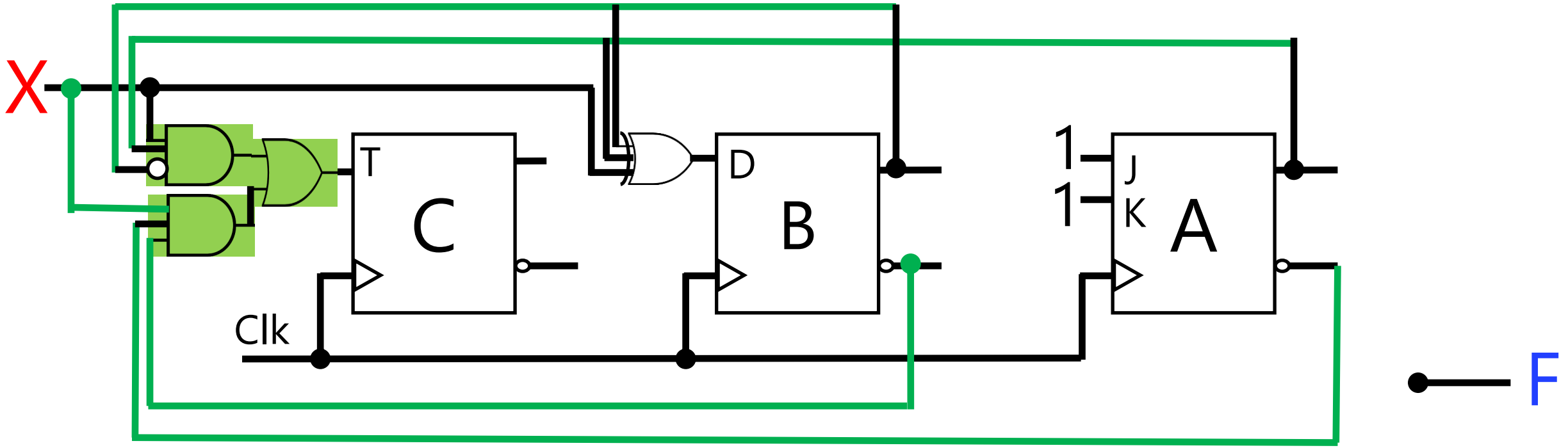
0. Do we need combinational logic or sequential logic? Do we need memory?
1. How many storage (flip-flops)? #FF
2. How many input and output?
3. Form the state (transition) diagram
4. Form the state table
5. Fill the state table
6. What type of storage (flip-flop)? RS, D, T, JK, or Mixed
7. Input (*excitation*) equations for each FF
8. Minimization of input (*excitation*) equations
9. Boolean function for the output
10. Minimization of output variable
11. Draw/Sketch Logic Circuit
12. (Optional) Test



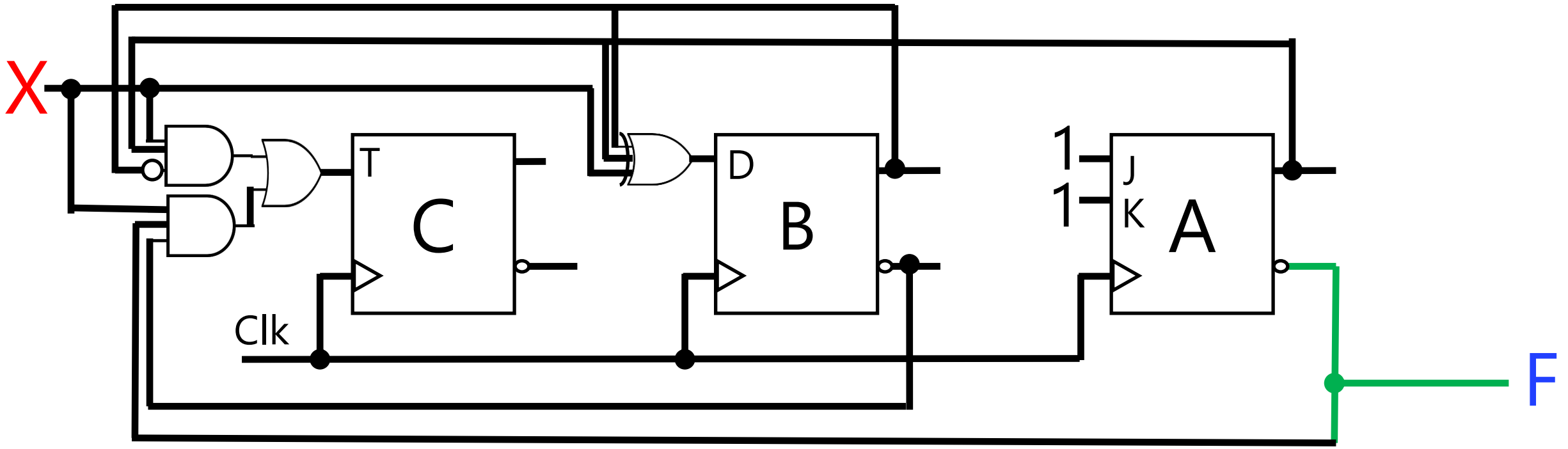
A	$J_A=1$	$K_A=1$
B	$D_B=F(X,C,B,A)=X \oplus B \oplus A$	
C	$T_C=F(X,C,B,A)=X'BA + XB'A'$	
F	$F(C,B,A) = A'$	



A	$J_A=1$	$K_A=1$
B	$D_B=F(X,C,B,A)=X \oplus B \oplus A$	
C	$T_C=F(X,C,B,A)=X'BA + XB'A'$	
F	$F(C,B,A) = A'$	

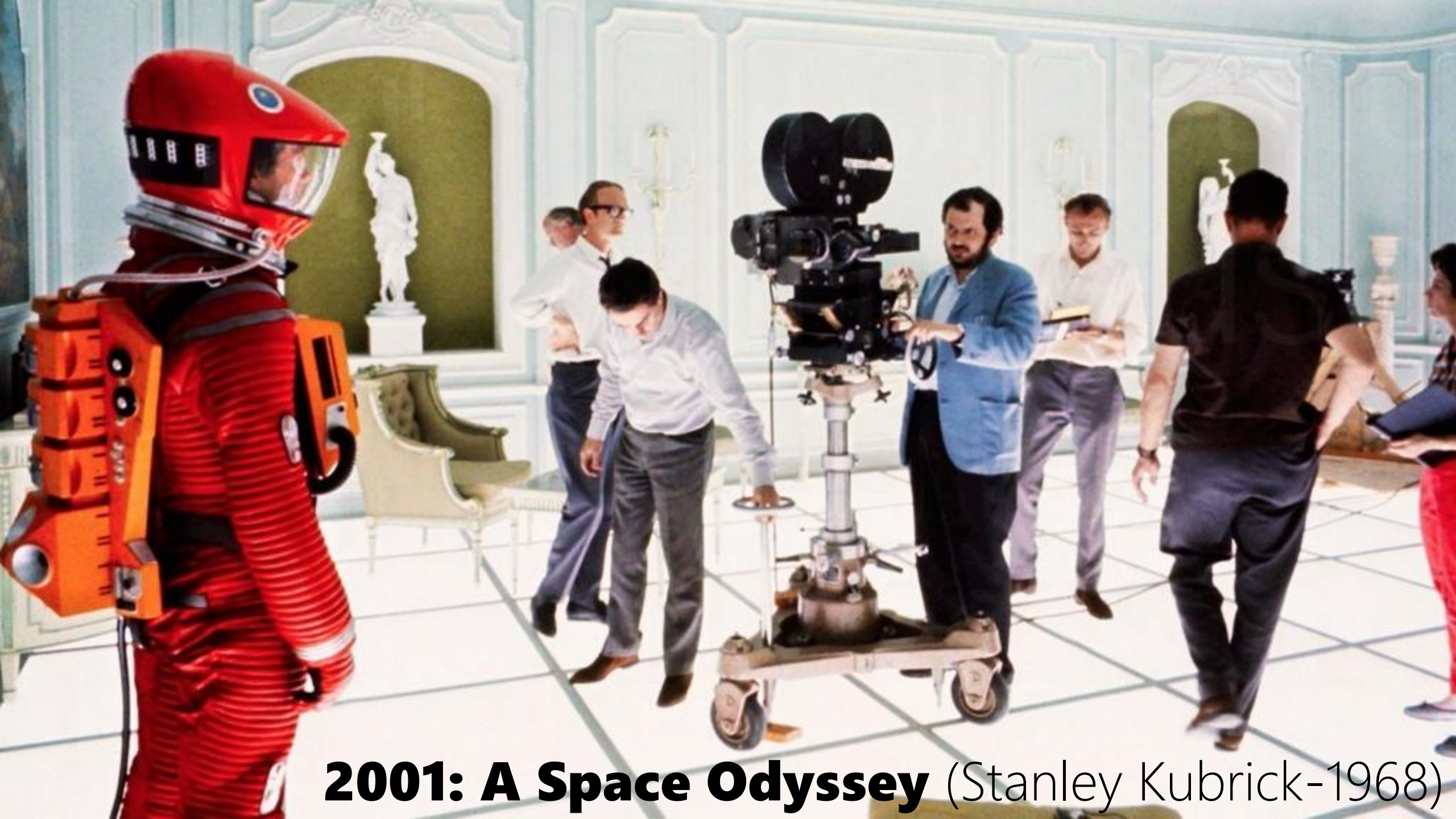


A	$J_A=1$	$K_A=1$
B	$D_B=F(X,C,B,A)=X \oplus B \oplus A$	
C	$T_C=F(X,C,B,A)=X'BA + XB'A'$	
F	$F(C,B,A) = A'$	



A	$J_A=1$	$K_A=1$
B	$D_B=F(X,C,B,A)=X \oplus B \oplus A$	
C	$T_C=F(X,C,B,A)=X'BA + XB'A'$	
F	$F(C,B,A) = A'$	

Synchronization
Practice Timing Diagram



2001: A Space Odyssey (Stanley Kubrick-1968)

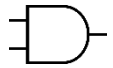
Number Systems |

| Computer Systems

Number Systems | $(12)_{10} \rightarrow (1100)_2$

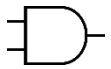
| Computer Systems

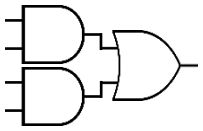
Number Systems | $(12)_{10} \rightarrow (1100)_2$

| Logic Gates | 

| Computer Systems

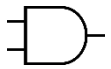
Number Systems | $(12)_{10} \rightarrow (1100)_2$

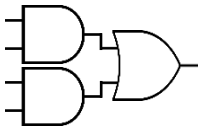
| Logic Gates | 

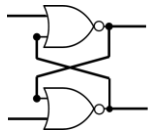
| Combinational Logic | 

| Computer Systems

Number Systems | $(12)_{10} \rightarrow (1100)_2$

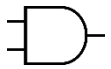
| Logic Gates | 

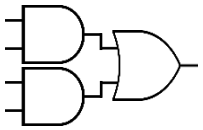
| Combinational Logic | 

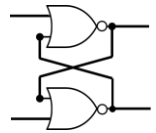
| Flip-Flops | 

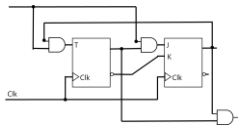
| Computer Systems

Number Systems | $(12)_{10} \rightarrow (1100)_2$

| Logic Gates | 

| Combinational Logic | 

| Flip-Flops | 

| Sequential Logic | 

| Computer Systems

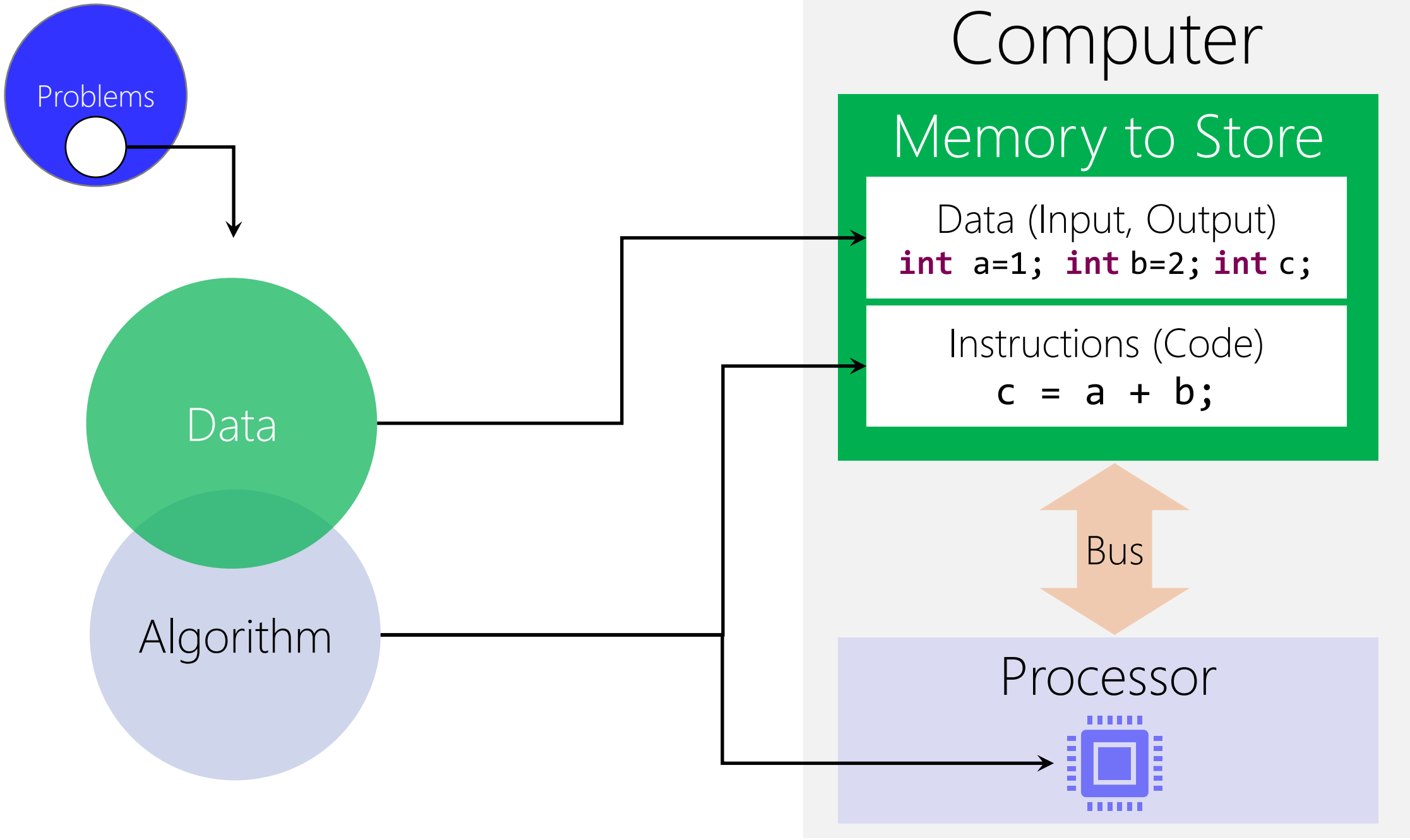
Design a Computer System

John von Neumann

(/vɒn ˈnɔɪmən/)
1903 –1957
Mathematician, Physicist, Computer Scientist, Engineer

Polymath
He integrated pure and applied sciences. He made major contributions to many fields, including:

- Mathematics
- Physics
- Economics (game theory)
- Computing
- Statistics


Problems
Data
Algorithm
Computer
Memory to Store
Data (Input, Output)
int a=1; int b=2; int c;
Instructions (Code)
c = a + b;
Bus
Processor

von Neumann Architecture

Principles

- Data and instructions are both stored in the main memory
- The content of the memory is addressable by location (regardless of what is stored in that location)
- Instructions are executed sequentially unless the order is explicitly modified

Human Natural Language	Type	Example
High Level Programing Language	Imperative (What)	SQL `SELECT * FROM Students`

Middle Level Programming Language	Declarative (What + How)	C, C++, Java, Python
		`int a = 1;`
		`a = a + 1;`

Low Level Programming Language (Assembly Language)		Assembly x86, Assembly Z80
		`MOV 1, AX`
		`INC AX`

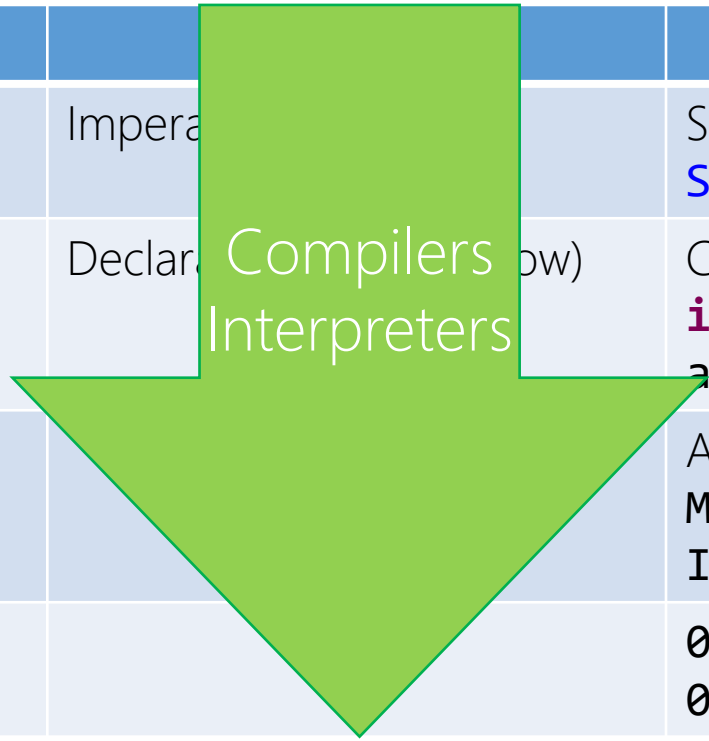
Binary Digits (Machine Language)		`00010011` `00001011`

Human Natural Language		Example
High Level Programing Language	Impera	SQL `SELECT * FROM Students`

Middle Level Programming Language	Declar ow)	C, C++, Java, Python
		`int a = 1;`
		`a = a + 1;`

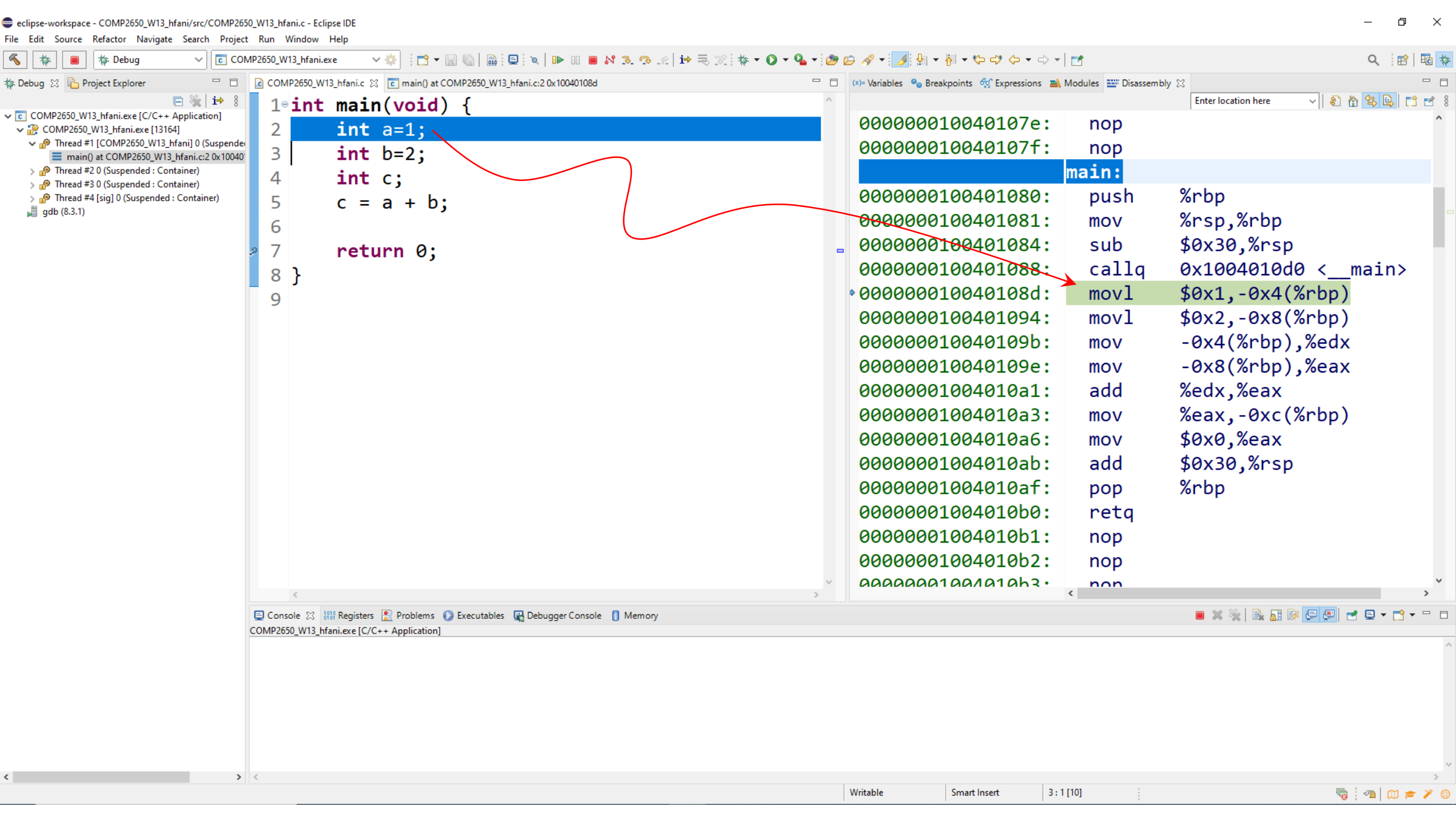
Low Level Programming Language (Assembly Language)		Assembly x86, Assembly Z80
		`MOV 1, AX`
		`INC AX`

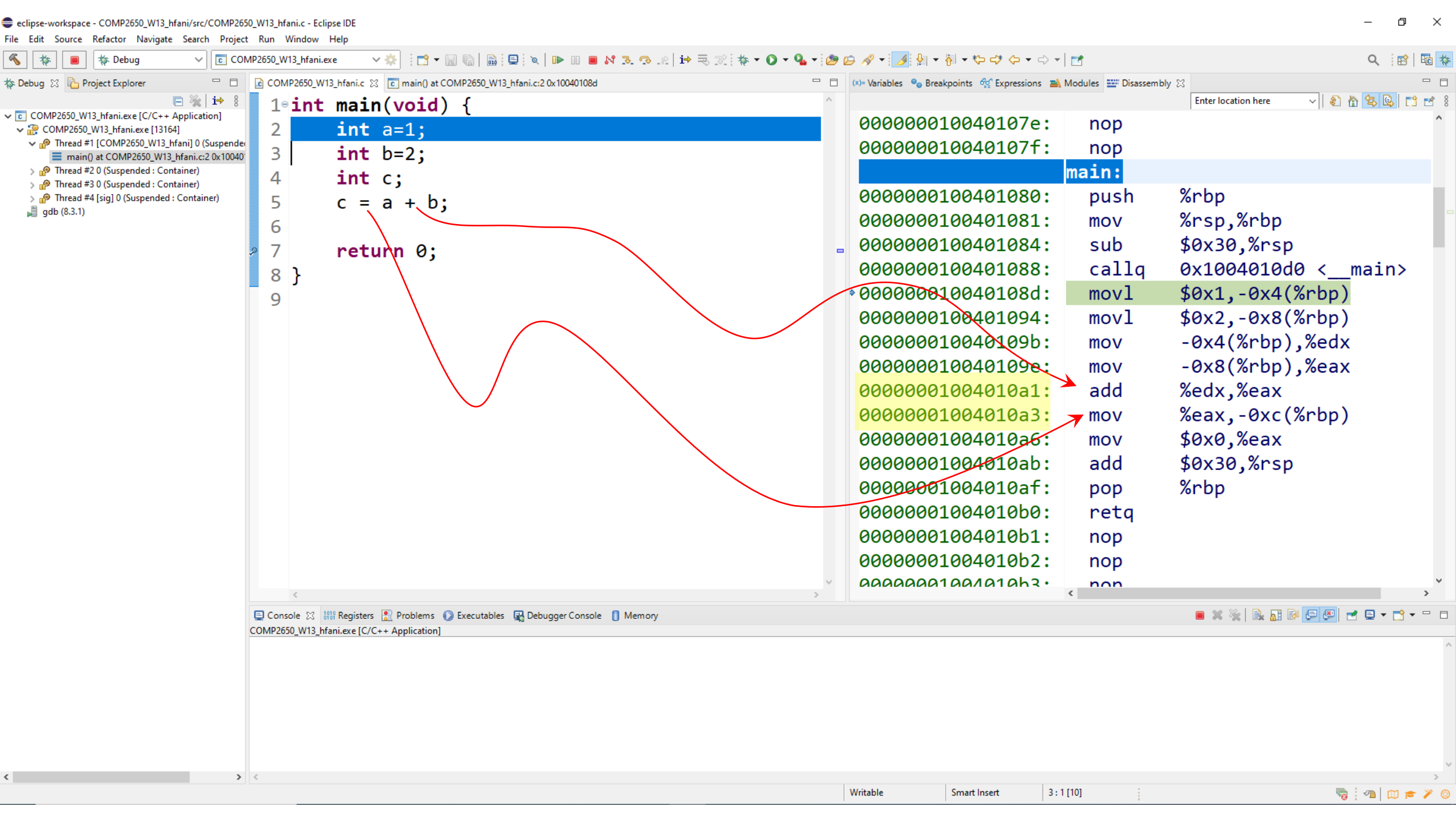
Binary Digits (Machine Language)		`00010011` `00001011`

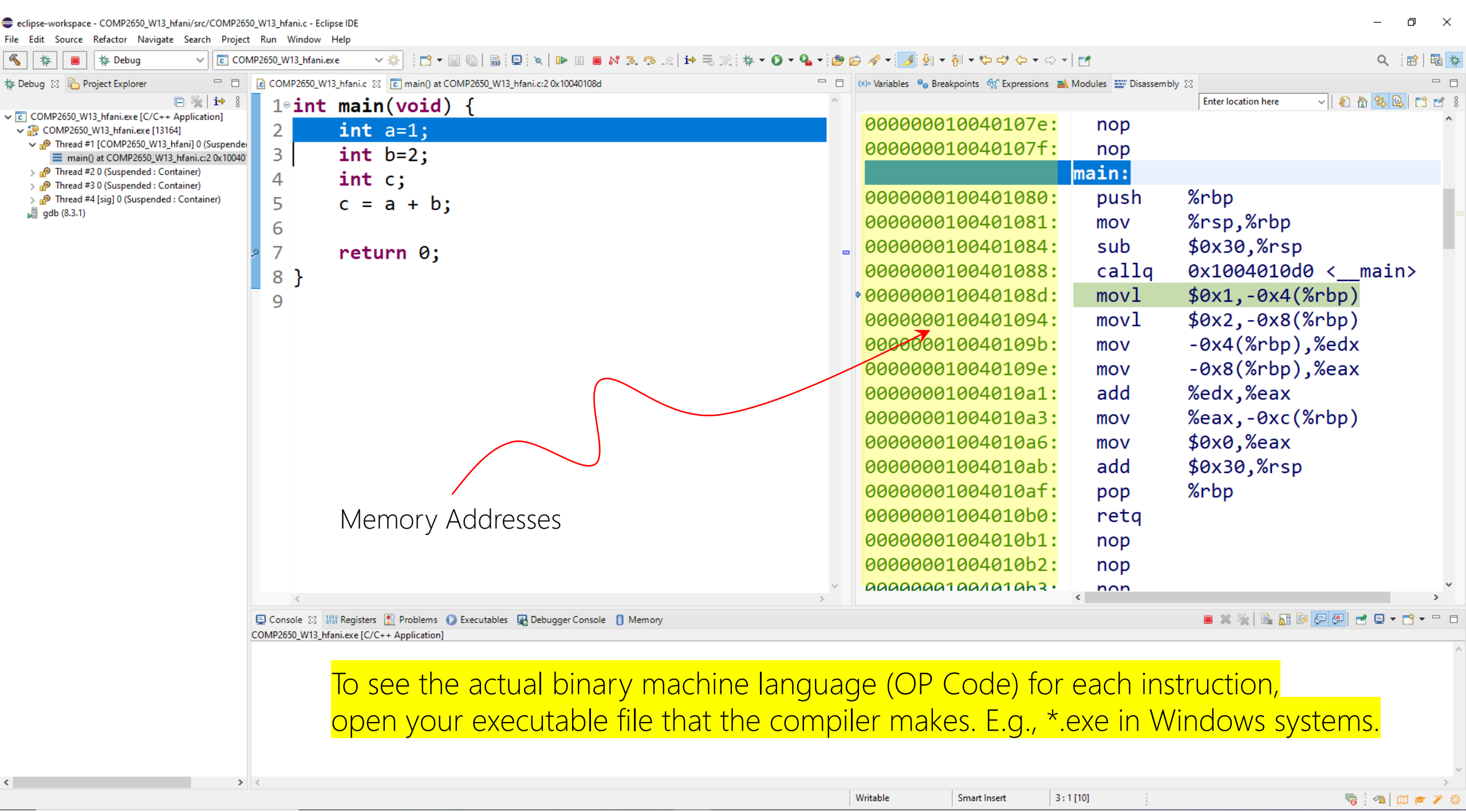


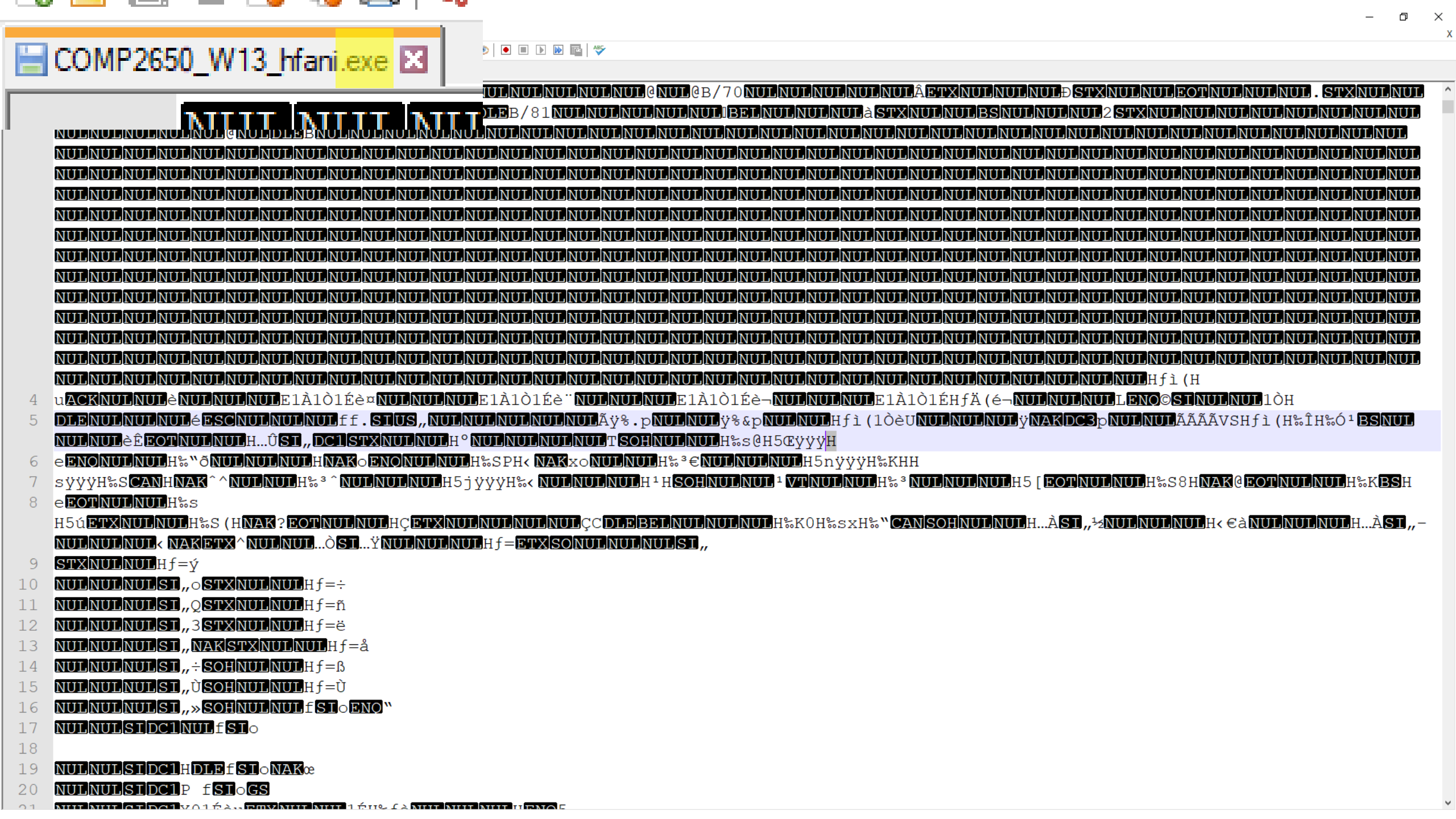
C/C++

Hossein's Computer


eclipse-workspace - COMP2650_W13_hfani/src/COMP2650_W13_hfani.c - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Debug
COMP2650_W13_hfani.exe
Debug
Project Explorer
COMP2650_W13_hfani.exe [C/C++ Application]
COMP2650_W13_hfani.exe [13164]
Thread #1 [COMP2650_W13_hfani] 0 (Suspende
main() at COMP2650_W13_hfani.c:2 0x10040
Thread #2 0 (Suspended : Container)
Thread #3 0 (Suspended : Container)
Thread #4 [sig] 0 (Suspended : Container)
gdb (8.3.1)
COMP2650_W13_hfani.c
main() at COMP2650_W13_hfani.c:2 0x10040108d
1 int main(void) {
2 int a=1;
3 int b=2;
4 int c;
5 c = a + b;
6
7 return 0;
8 }
9
Variables
Breakpoints
Expressions
Modules
Disassembly
Enter location here
000000010040107e: nop
000000010040107f: nop
main:
0000000100401080: push %rbp
0000000100401081: mov %rsp,%rbp
0000000100401084: sub $0x30,%rsp
0000000100401088: callq 0x1004010d0 <__main>
000000010040108d: movl $0x1,-0x4(%rbp)
0000000100401094: movl $0x2,-0x8(%rbp)
000000010040109b: mov -0x4(%rbp),%edx
000000010040109e: mov -0x8(%rbp),%eax
00000001004010a1: add %edx,%eax
00000001004010a3: mov %eax,-0xc(%rbp)
00000001004010a6: mov $0x0,%eax
00000001004010ab: add $0x30,%rsp
00000001004010af: pop %rbp
00000001004010b0: retq
00000001004010b1: nop
00000001004010b2: nop
Console
Registers
Problems
Executables
Debugger Console
Memory
COMP2650_W13_hfani.exe [C/C++ Application]
Writable
Smart Insert
3 : 1 [10]


eclipse-workspace - COMP2650_W13_hfani/src/COMP2650_W13_hfani.c - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Debug
COMP2650_W13_hfani.exe
Debug
Project Explorer
COMP2650_W13_hfani.exe [C/C++ Application]
COMP2650_W13_hfani.exe [13164]
Thread #1 [COMP2650_W13_hfani] 0 (Suspende
main() at COMP2650_W13_hfani.c:2 0x10040
Thread #2 0 (Suspended : Container)
Thread #3 0 (Suspended : Container)
Thread #4 [sig] 0 (Suspended : Container)
gdb (8.3.1)
COMP2650_W13_hfani.c
main() at COMP2650_W13_hfani.c:2 0x10040108d
1 int main(void) {
2 int a=1;
3 int b=2;
4 int c;
5 c = a + b;
6
7 return 0;
8 }
9
Variables
Breakpoints
Expressions
Modules
Disassembly
Enter location here
000000010040107e: nop
000000010040107f: nop
main:
0000000100401080: push %rbp
0000000100401081: mov %rsp,%rbp
0000000100401084: sub $0x30,%rsp
0000000100401088: callq 0x1004010d0 <__main>
000000010040108d: movl $0x1,-0x4(%rbp)
0000000100401094: movl $0x2,-0x8(%rbp)
000000010040109b: mov -0x4(%rbp),%edx
000000010040109e: mov -0x8(%rbp),%eax
00000001004010a1: add %edx,%eax
00000001004010a3: mov %eax,-0xc(%rbp)
00000001004010a6: mov $0x0,%eax
00000001004010ab: add $0x30,%rsp
00000001004010af: pop %rbp
00000001004010b0: retq
00000001004010b1: nop
00000001004010b2: nop
Console
Registers
Problems
Executables
Debugger Console
Memory
COMP2650_W13_hfani.exe [C/C++ Application]
Writable
Smart Insert
3 : 1 [10]


eclipse-workspace - COMP2650_W13_hfani/src/COMP2650_W13_hfani.c - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Debug
COMP2650_W13_hfani.exe
Debug
Project Explorer
COMP2650_W13_hfani.exe [C/C++ Application]
COMP2650_W13_hfani.exe [13164]
Thread #1 [COMP2650_W13_hfani] 0 (Suspende
main() at COMP2650_W13_hfani.c:2 0x10040
Thread #2 0 (Suspended : Container)
Thread #3 0 (Suspended : Container)
Thread #4 [sig] 0 (Suspended : Container)
gdb (8.3.1)
COMP2650_W13_hfani.c
main() at COMP2650_W13_hfani.c:2 0x10040108d
1 int main(void) {
2 int a=1;
3 int b=2;
4 int c;
5 c = a + b;
6
7 return 0;
8 }
9
Memory Addresses
Variables
Breakpoints
Expressions
Modules
Disassembly
Enter location here
000000010040107e: nop
000000010040107f: nop
main:
0000000100401080: push %rbp
0000000100401081: mov %rsp,%rbp
0000000100401084: sub $0x30,%rsp
0000000100401088: callq 0x1004010d0 <__main>
000000010040108d: movl $0x1,-0x4(%rbp)
0000000100401094: movl $0x2,-0x8(%rbp)
000000010040109b: mov -0x4(%rbp),%edx
000000010040109e: mov -0x8(%rbp),%eax
00000001004010a1: add %edx,%eax
00000001004010a3: mov %eax,-0xc(%rbp)
00000001004010a6: mov $0x0,%eax
00000001004010ab: add $0x30,%rsp
00000001004010af: pop %rbp
00000001004010b0: retq
00000001004010b1: nop
00000001004010b2: nop
Console
Registers
Problems
Executables
Debugger Console
Memory
COMP2650_W13_hfani.exe [C/C++ Application]
To see the actual binary machine language (OP Code) for each instruction,
open your executable file that the compiler makes. E.g., *.exe in Windows systems.
Writable
Smart Insert
3 : 1 [10]


COMP2650_W13_Hfani.exe
NULL NULL NUL

```
hfani@alpha:~$ hexdump main.o
0000000 457f 464c 0102 0001 0000 0000 0000 0000
0000010 0001 003e 0001 0000 0000 0000 0000 0000
0000020 0000 0000 0000 0000 0208 0000 0000 0000
0000030 0000 0000 0040 0000 0000 0040 000b 000a
0000040 4855 e589 45c7 01fc 0000 9000 c35d 4700
0000050 4343 203a 4428 6265 6169 206e 3031 322e
0000060 312e 362d 2029 3031 322e 312e 3220 3230
0000070 3031 3131 0030 0000 0014 0000 0000 0000
0000080 7a01 0052 7801 0110 0c1b 0807 0190 0000
0000090 001c 0000 001c 0000 0000 0000 000e 0000
00000a0 4100 100e 0286 0d43 4906 070c 0008 0000
00000b0 0000 0000 0000 0000 0000 0000 0000 0000
00000c0 0000 0000 0000 0000 0001 0000 0004 fff1
00000d0 0000 0000 0000 0000 0000 0000 0000 0000
00000e0 0000 0000 0003 0001 0000 0000 0000 0000
00000f0 0000 0000 0000 0000 0000 0000 0003 0002
0000100 0000 0000 0000 0000 0000 0000 0000 0000
0000110 0000 0000 0003 0003 0000 0000 0000 0000
0000120 0000 0000 0000 0000 0000 0000 0003 0005
0000130 0000 0000 0000 0000 0000 0000 0000 0000
```

Instruction	Operation Code (OP Code)
c = a + b	000 XXXX YYYY ZZZZ
c = a – b	001 XXXX YYYY ZZZZ
c = a / b	010 XXXX YYYY ZZZZ
c = a * b	011 XXXX YYYY ZZZZ
c = {number}	100 XXXX XXX XXXX
…	…

Instruction Encoders very simplified version!

Design Processor

Processor can only see binary-coded instructions

Processor only understand machine language

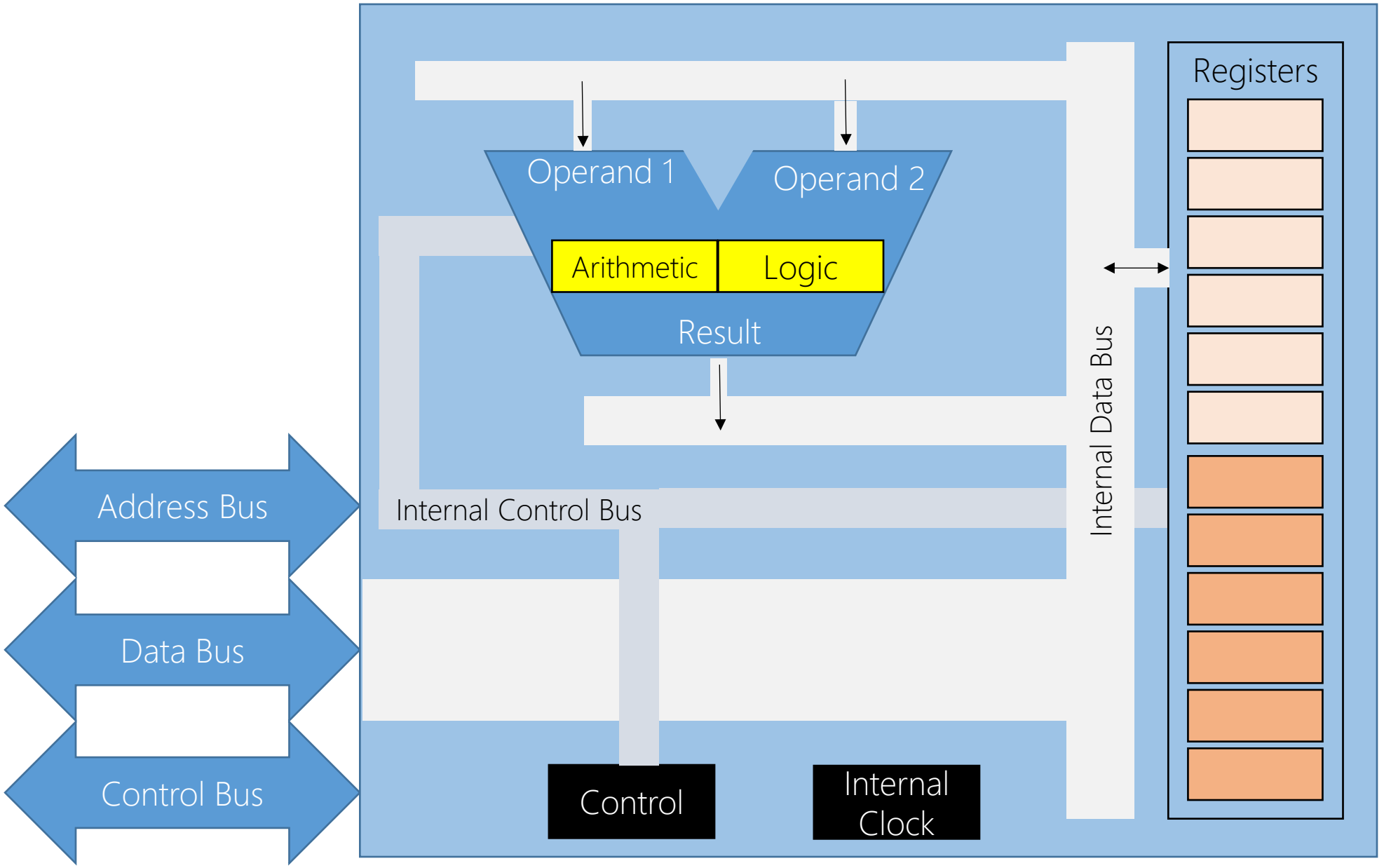
Instruction Set →Instruction Decoder

Operation Code (OP Code)	What to do?
000 XXXX YYYY ZZZZ	1) Fetch the first operand from memory at XXXX address 2) Store the first operand inside somewhere (AX) 3) Fetch the second operand from memory at YYYY address 4) Store the first operand inside somewhere else (BX) 5) Use the n-bit Adder to add AX and BX 6) Store the result inside somewhere else (CX) 7) Push CX to memory at ZZZZ address
001 XXXX YYYY ZZZZ	
010 XXXX YYYY ZZZZ	
011 XXXX YYYY ZZZZ	
100 XXXX XXX XXXX	
…	

Instruction Decoder
very simplified version!

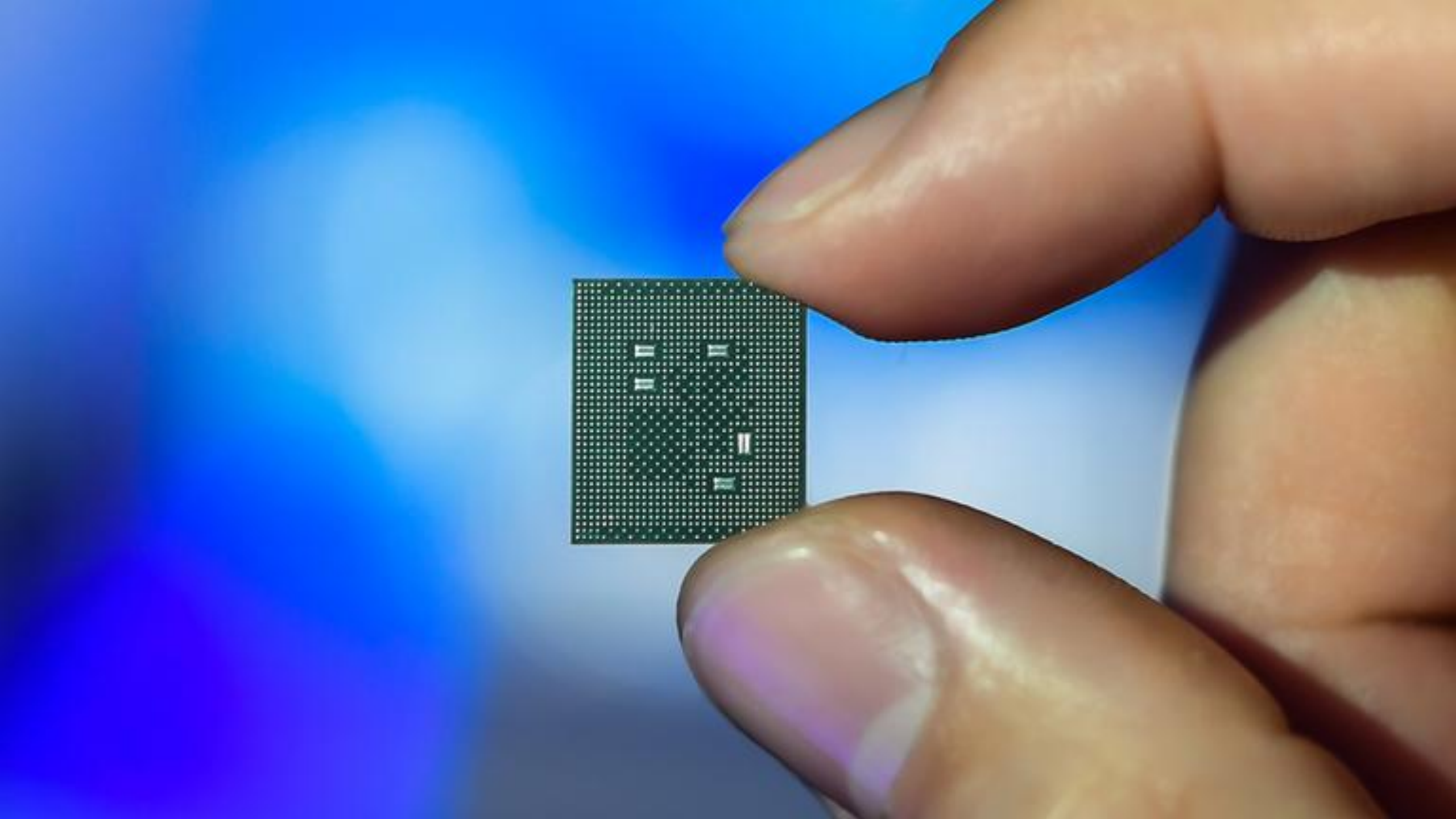
Operation Code (OP Code)	What to do?
000 XXXX YYYY ZZZZ	1) Fetch the first operand from memory at XXXX address 2) Store the first operand inside somewhere (AX) 3) Fetch the second operand from memory at YYYY address 4) Store the first operand inside somewhere else (BX) 5) Use the n-bit Adder to add AX and BX 6) Store the result inside somewhere else (CX) 7) Push CX to memory at ZZZZ address
001 XXXX YYYY ZZZZ	
010 XXXX YYYY ZZZZ	
011 XXXX YYYY ZZZZ	
100 XXXX XXX XXXX	
…	

Instruction Decoder
very simplified version!



City

- Streets
  - 1-way
  - 2-way
- Gates
- Bridges
- Warehouses
- Policing



COMP-2660
Computer Architecture II
Microprocessor Programming


Starchild: **qbit**